

BÁO CÁO KỸ THUẬT CHI TIẾT

DUPLICATE VÀ CONFLICT TRONG HỆ THỐNG AI RAG

Phân tích thuật toán, luồng so sánh, vị trí lưu dữ liệu, cơ chế đồng thời, structured facts, retrieval, citation và bằng chứng kiểm thử bám sát repository

Phạm vi audit: upload → ingestion → canonical identity → chunk dedup → embedding → structured conflict → persistence → retrieval → generation → review

Repository: agentic-rag-batch-2-g1-develop

Phiên bản báo cáo: 1.0

Ngày chốt audit: 06/08/2026

Tuyên bố phạm vi và mức bằng chứng

Báo cáo này mô tả hành vi có thể truy vết đến code, migration và test trong repository. Nó không coi một thiết kế trên giấy là đã triển khai, cũng không coi static SQL contract là bằng chứng migration đã chạy trên production. Mỗi kết luận được phân loại theo ba mức sau.

Mức	Ý nghĩa	Được phép kết luận
A — Có trong code	Đường gọi và invariant tồn tại trong mã nguồn/migration.	Có thể giải thích chính xác code dự định làm gì.
B — Test cục bộ	Unit/integration/E2E với fake adapter hoặc static migration contract đã chạy đạt.	Có thể nói hành vi đã được kiểm tra trong môi trường local của repo.
C — Live verified	Migration/RLS/RPC/concurrency đã chạy trên Supabase/PostgreSQL/Qdrant thật.	Chỉ được khẳng định khi có log hoặc test staging/live riêng.

Giới hạn trung thực

Audit hiện đạt mức A và B cho phần lớn duplicate/conflict. Chưa có bằng chứng trong lần kiểm tra này rằng migration 08/09/10/16 đã được apply và race/RLS đã được thử trên Supabase staging hoặc production thật.

Tóm tắt điều hành

Hệ thống không giải duplicate/conflict bằng một phép cosine similarity duy nhất. Nó dùng nhiều lớp, mỗi lớp có identity và quyền tự động riêng. Exact identity mới được phép early-return, auto-alias hoặc reuse vector. Fuzzy similarity chỉ sinh candidate và phải được kiểm tra lại bằng text/claim/scope. Với bằng nghiệp vụ, hệ thống chuyển toàn bộ row thành structured claims rồi diff theo khóa nghiệp vụ, qualifier và thời gian; chỉ trường hợp thật sự comparable, thời gian giao nhau và value khác mới thành conflict_candidate.

Hạng mục	Tình trạng	Cơ chế chính	Bằng chứng / khoảng trống
Byte-exact duplicate	Đã triển khai	SHA-256 raw bytes; owner+notebook scope; partial unique index; retry winner	99 test duplicate tập trung đạt; chưa live race DB
Cross-format exact duplicate	Đã triển khai	Typed canonical projection; stable JSON; strict SHA-256; trust gate	TXT/DOCX/MD/HTML và CSV/XLSX/DOCX/HTML có test
Chunk exact dedup	Đã triển khai	Strict hash; embedding checksum; same-model vector reuse; same-batch reuse	Provider chỉ nhận chunk chưa reuse trong unit test
Near duplicate	Đã triển khai bảo thủ	SimHash-LSH/ANN tạo candidate; full analyzer xác minh	Không fuzzy reuse, không auto-delete

Hạng mục	Tình trạng	Cơ chế chính	Bằng chứng / khoảng trống
Generic text conflict	Đã triển khai	Claim extraction; quantity/date/unit/negation/modality/scope checks	Phù hợp prose; không thay structured table diff
Structured table conflict	Đã triển khai	Full-row analyzer; multi-facet scope; qualifiers; temporal; authority; O(n+m) diff	101 test tập trung đạt; chưa live migration 16
Retrieval/citation/review	Đã triển khai	Structured-first; fallback vector; relation-id citation pair; optimistic review	84 test audit đạt; service-level skip-vector còn chủ yếu chứng minh bằng control flow

Invariant cốt lõi

Exact có thể tự động. Fuzzy chỉ là candidate. Conflict chưa giải quyết phải giữ cả hai phía và dẫn nguồn cả hai. Mọi quyết định phải tenant-scoped, có version detector và có khả năng audit/rollback.

Thuật ngữ cần hiểu trước khi đọc

Thuật ngữ	Giải thích trong repo này
Duplicate	Hai document/chunk biểu diễn cùng nội dung theo một identity đã định nghĩa; không đồng nghĩa với 'trông hơi giống'.
Canonical	Biểu diễn chuẩn, ổn định để so sánh; hoặc document gốc được chọn làm đại diện của một nhóm exact duplicate.
Raw hash	SHA-256 tính trực tiếp trên bytes file; đổi một byte thì hash thay đổi.
Canonical hash	SHA-256 tính trên JSON typed projection sau parse; có thể giống nhau dù định dạng file khác.
Alias document	Document nguồn được đánh dấu trỏ tới canonical_document_id; vẫn giữ metadata/audit thay vì xóa lịch sử.
Near duplicate	Nội dung gần giống nhưng chưa đủ điều kiện exact; chỉ được gắn cờ để review.
Claim	Một mệnh đề có subject, predicate, value, scope, qualifier, temporal, authority và provenance.
Conflict candidate	Hai claim đã comparable, có hiệu lực chồng lấn nhưng giá trị khác; chờ human review.
Qualifier	Điều kiện làm thay đổi nghĩa của value, ví dụ loại giá, cơ sở giá, phương án thanh toán, VAT.
Business scope	Phạm vi nghiệp vụ nhiều facet: dự án/tòa/căn, sản phẩm, phân khúc, kênh, khách hàng...
Effective time	Khoảng thời gian dữ liệu có hiệu lực; khác với thời điểm xuất bản hoặc ingest.
Generation token	UUID mới mỗi lần worker claim/reclaim job; dùng fencing để worker cũ không commit sau khi mất lease.
LSH	Locality-Sensitive Hashing: chia SimHash thành band để tìm candidate nhanh; không phải quyết định merge.
MMR	Maximal Marginal Relevance: cân bằng relevance và diversity khi chọn context retrieval.

Mục lục điều hướng

[1. Kiến trúc và vị trí dữ liệu](#)

[2. Duplicate — thuật toán và luồng chi tiết](#)

[3. Conflict — từ prose đến structured facts](#)

[4. Luồng tương tác và lưu tạm/lưu bền](#)

[5. Cách chứng minh, giới hạn và deployment gate](#)

[Phụ lục A. Evidence catalog](#)

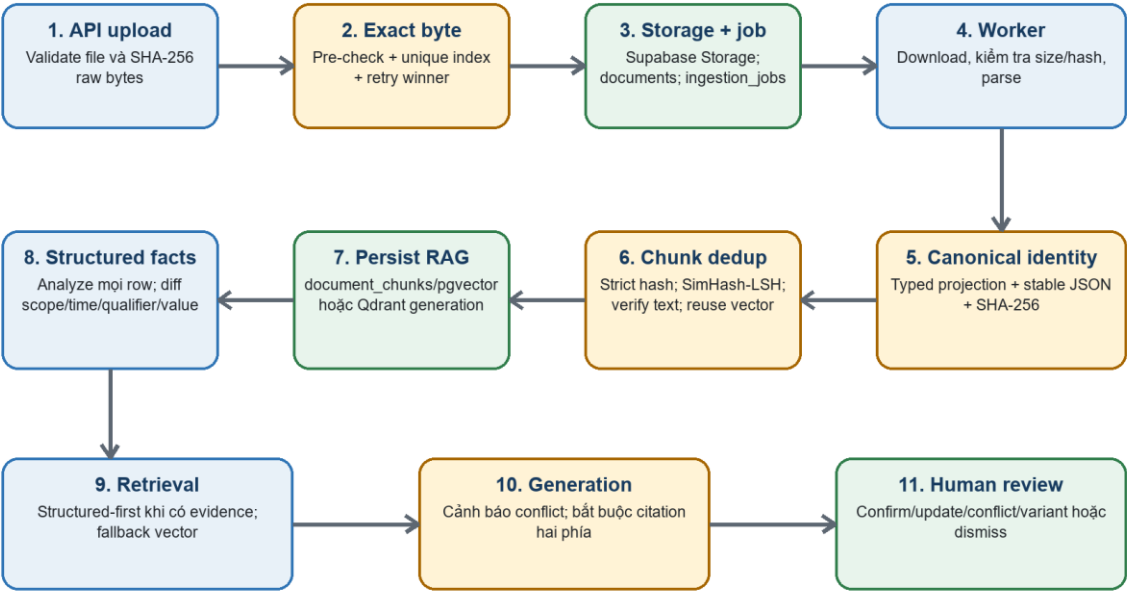
[Phụ lục B. Kịch bản bảo vệ và hỏi–đáp](#)

1. Kiến trúc và vị trí dữ liệu

Luồng xử lý được chia thành API upload, storage/metadata, ingestion worker, pipeline parse–chunk–embed, knowledge-quality, structured-facts và chat/review. Duplicate được chặn càng sớm càng tốt để giảm chi phí; conflict chỉ được xác định sau khi có đủ ngữ cảnh so sánh.

Luồng tổng thể: từ upload đến trả lời

Màu xanh: xử lý; màu lục: lưu bền; màu vàng: quyết định bảo thủ



Hình 1 — Luồng end-to-end và vị trí các quyết định duplicate/conflict.

1.1. Các đối tượng dữ liệu chính

Đối tượng	Sinh ở đâu	Dùng để làm gì	Vị trí giữ
ValidatedDocumentFile	validate_document_file	Tên đã sanitize, MIME suy từ extension, size, raw hash, bytes	RAM của API request
Document metadata	DocumentService	Owner/notebook, object path, hash, status, canonical/version	PostgreSQL documents
Ingestion job	enqueue RPC	Profile, attempt, lease, worker_id, claim_token	PostgreSQL ingestion_jobs
DocumentSource	Worker sau download	Bytes + tenant + document/version + generation	RAM worker
ParsedDocument	Pipeline.prepare	Text/pages/elements/tables/images/warnings/confidence	RAM worker

Đối tượng	Sinh ở đâu	Dùng để làm gì	Vị trí giữ
DocumentFingerprint	content_identity + analysis	Strict hash, loose signature, identity version, trust signals	RAM; các fingerprint cần thiết được persist vào document/metadata
ChunkDedupProbe/Plan	chunk_preembedding	Candidate lookup, exact/fuzzy decision, vector reuse plan	RAM; decision metadata persist theo chunk
Embedded chunks	pipeline.embed	Text, vector, provenance và quality metadata	RAM trước commit; PostgreSQL/pgvector hoặc Qdrant sau commit/staging
TableAnalysis	structured table analyzer	Schema, mọi row claim, confidence/warnings	RAM; không lưu raw table dump thứ hai
Structured persistence batch	persistence.py	Snapshots, claims, source cells, provenance	RAM trước atomic RPC; PostgreSQL sau RPC
Claim relation	table diff/comparison	unchanged/updated/variant/conflict/uncertain...	PostgreSQL claim_relations + audit

1.2. Hệ thống so sánh tài liệu mới với tài liệu cũ như thế nào?

Không có một bước 'đọc lại toàn bộ file cũ rồi so từng ký tự' cho mọi upload. Hệ thống lưu các identity và claim đã chuẩn hóa để lần sau lookup nhanh, nhưng vẫn xác minh lại ở những chỗ fuzzy có nguy cơ false positive.

Mức so sánh	Phía mới	Phía cũ được lấy từ đâu	Khóa/điều kiện
Raw file	SHA-256 trên bytes request	documents.content_hash	owner_id + notebook_id + content_hash; canonical active, không failed
Canonical document	Parse → typed JSON → strict hash/version	documents.normalized_content_hash + normalization_version	cùng tenant, ready/active/canonical; trust gate
Chunk	Strict hash, SimHash, embedding checksum	document_chunks fingerprint/vector metadata	exact hash hoặc trùng LSH band; application verify full text
Generic document relation	Một số probe chunk/vector + claim signals	Vector index candidates + target text metadata	tenant/document scope; coverage và critical-difference checks
Structured table	Toàn bộ current claims	Prior structured_claims từ document đủ điều kiện	table family + subject/predicate + stable qualifier; diff scope/time/value

Trả lời ngắn cho câu hỏi 'có lưu tạm không?'

Có dữ liệu tạm trong RAM: bytes, ParsedDocument, canonical JSON, probes, plan, embeddings và relation payload. Không có bảng DB tạm chuyên để so sánh. Dữ liệu bền được ghi vào Storage/PostgreSQL/vector index. Với Qdrant, generation mới được ghi theo claim_token trước DB completion; đó là staging bền có fencing/reconciliation, không phải file tạm.

1.3. Phân biệt duplicate, version, variant, update và conflict

Khái niệm	Điều kiện điển hình	Hành động đúng
Exact duplicate	Cùng bytes hoặc cùng strict canonical identity đáng tin	Reuse/alias tự động; không embed lại
Near duplicate	Rất giống nhưng không exact, không có critical difference đã xác nhận	Giữ cả hai; pending review; không reuse fuzzy
Version/update	Cùng đối tượng nhưng effective interval nối tiếp/không chồng lấn	Giữ lịch sử; relation updated/supersedes
Conditional variant	Khác scope hoặc qualifier rõ ràng	Giữ cả hai như các điều kiện khác nhau
Conflict candidate	Comparable + time overlap + value khác vượt tolerance	Giữ cả hai, pending human review, cite hai phía
Uncertain	Thiếu scope/time/qualifier hoặc confidence thấp/khóa row mơ hồ	Không tự quyết; giữ evidence và review
Distinct	Không đủ evidence cùng identity/nội dung	Xử lý như hai nguồn độc lập

2. Duplicate — thuật toán và luồng xử lý chi tiết

Duplicate được xử lý theo bốn lớp. Lớp càng sớm càng rẻ nhưng identity càng hẹp; lớp càng muộn hiểu nhiều semantics hơn nhưng đã tốn parse hoặc embedding. Thiết kế đúng không cố ép một thuật toán giải mọi trường hợp.

Bốn lớp duplicate không thay thế lẫn nhau

Mỗi lớp có identity, chi phí và quyền tự động khác nhau

Lớp 1 — Raw bytes	SHA-256 toàn bộ bytes Bước: Trước Storage	Quyết định: Được early-return tự động
Lớp 2 — Canonical document	Typed projection → stable JSON → SHA-256 Bước: Sau parse	Quyết định: Chỉ auto-alias khi trust gate đạt
Lớp 3 — Chunk exact	Strict SHA-256 + embedding input checksum Bước: Trước embedding	Quyết định: Chỉ reuse cùng model + cùng input
Lớp 4 — Near duplicate	SimHash-LSH/ANN sinh candidate + full verification Bước: Review signal	Quyết định: Không auto-delete, không fuzzy reuse

Hình 2 — Bốn lớp duplicate và quyền tự động tương ứng.

2.1. Bước đầu: nhận và validate file cụ thể ra sao?

FastAPI nhận danh sách UploadFile nhưng domain service không tin client-supplied MIME để quyết định nội dung. Mỗi file được đọc thành bytes và đi qua validate_document_file. API giới hạn tối đa 20 file/lần; domain giới hạn 10 MiB/file.

Kiểm tra	Cách làm trong code	Lỗi bị chặn	Tác dụng với duplicate
Số file	Router từ chối khi len(files) > 20	Upload batch quá lớn	Tránh chiếm tài nguyên trước hashing/parse
Filename	Lấy basename; bắt buộc 1–255 ký tự	Tên rỗng/path bất thường/quá dài	Tên chỉ phục vụ metadata; không tham gia hash
Storage filename	NFKC; chỉ giữ alnum/./_ ; có underscore; giới hạn 200 ký tự	Path traversal/ký tự nguy hiểm	Không biến tên thành identity
Không rỗng	if not content	File zero byte	Không tạo hash/record vô nghĩa
Kích thước	len(content) ≤ 10 × 1024 × 1024	File quá 10 MiB	Giới hạn chi phí RAM/hash/parser

Kiểm tra	Cách làm trong code	Lỗi bị chặn	Tác dụng với duplicate
Định dạng	Extension thuộc PDF/DOCX/PPTX/XLSX/CSV/MD/HTML/TXT	Loại không hỗ trợ	Bảo đảm parser contract có thể áp dụng
PDF signature	bytes phải bắt đầu bằng %PDF-	Đổi đuôi giả PDF	Không hash/store nội dung giả dạng
Office archive	Mở ZIP; cần [Content_Types].xml và word/, ppt/ hoặc xl/	DOCX/PPTX/XLSX hỏng hoặc đổi đuôi	Tăng độ tin cậy extraction/canonical
Text encoding	decode UTF-8-SIG	CSV/MD/HTML/TXT encoding không hỗ trợ	Tránh canonical khác do decode mơ hồ
Raw hash	sha256(content).hexdigest()	Không phải validation error	Sinh exact byte identity ngay trong cùng bước

```
ValidatedDocumentFile(
    original_filename = basename(filename),
    storage_filename = sanitize(original_filename),
    mime_type         = MIME_TYPE_BY_EXTENSION[extension],
    size_bytes        = len(content),
    content_hash       = SHA256(content),
    content            = content,
)
```

MIME type được hiểu đúng như thế nào?

Service không nhận mime_type trong công thức identity. MIME được suy từ extension đã whitelist, sau đó signature/archive/encoding được kiểm tra. Điều này không phải antivirus hay content-disarm; báo cáo không khẳng định ngoài phạm vi code hiện có.

2.2. Kỹ thuật 1 — SHA-256 raw bytes

2.2.1. Công thức và ý nghĩa

```
H_raw      = SHA256(file_bytes)
lookupKey = (owner_id, notebook_id, H_raw)
```

SHA-256 nhận chuỗi bytes dài tùy ý và trả về digest 256 bit, thường biểu diễn bằng 64 ký tự hex. Một thay đổi nhỏ ở bytes tạo digest rất khác (avalanche effect). Hệ thống dùng hash như khóa identity chính xác, không dùng nó để đo độ gần giống. Xác suất collision về mặt thực tế cực thấp, nhưng code vẫn có guard ở mức chunk: nếu strict hash giống mà normalized text khác thì fail closed thay vì merge.

Thành phần	Có nằm trong hash?	Vai trò
Raw bytes	Có	Xác định byte-exact identity

Thành phần	Có nằm trong hash?	Vai trò
Filename	Không	Có thể đổi tên mà vẫn nhận ra cùng file
MIME/extension	Không	Dùng validate/routing parser, không dùng làm content identity
owner_id	Không	Là scope query/unique; tenant A không ảnh hưởng tenant B
notebook_id	Không	Là scope notebook; cùng file ở notebook khác vẫn được phép có canonical riêng

Không băm owner/notebook chung với bytes là lựa chọn có chủ đích. Content hash mô tả nội dung; tenant scope mô tả nơi phép so sánh có quyền nhìn. Tách hai khái niệm giúp kiểm tra integrity bằng đúng H_raw ở worker và cho database đánh index composite rõ ràng.

2.2.2. Luồng lookup và early-return

1. API/domain validate file và tính H_raw trong RAM.
2. Repository query documents với owner_id, notebook_id, content_hash; chỉ nhận document active, không failed và canonical_document_id IS NULL.
3. Kết quả được sắp created_at, id tăng dần để chọn canonical ổn định khi cần sửa dữ liệu cũ.
4. Nếu tìm thấy: DocumentService trả UploadOutcome(document=existing, is_duplicate=true).
5. Return xảy ra trước create_uploading, Supabase Storage upload và enqueue ingestion; vì vậy không parse, không chunk, không contextualize và không embed.
6. Nếu chưa thấy: tạo document_id/object path, insert metadata trạng thái uploading, upload object rồi enqueue job.

Kết quả lưu trữ ở lớp 1

Duplicate cùng bytes được phát hiện ở pre-check hoặc thua unique race đều không tạo object mới. Service trả document canonical đã có. Đây là tiết kiệm mạnh nhất vì dừng trước Storage và ingestion.

2.2.3. Worker tính lại hash để kiểm tra integrity

Sau khi job được claim, worker download object từ storage và gọi _verify_download. Nó so len(content) với size_bytes đã lưu và so SHA256(content) với documents.content_hash. Mục tiêu ở đây không phải tìm duplicate lần nữa mà là phát hiện object bị thiếu byte/thay đổi giữa upload và ingestion.

```
if len(downloaded_bytes) != job.size_bytes: fail
if SHA256(downloaded_bytes) != job.content_hash: fail
else: build DocumentSource and continue parsing
```

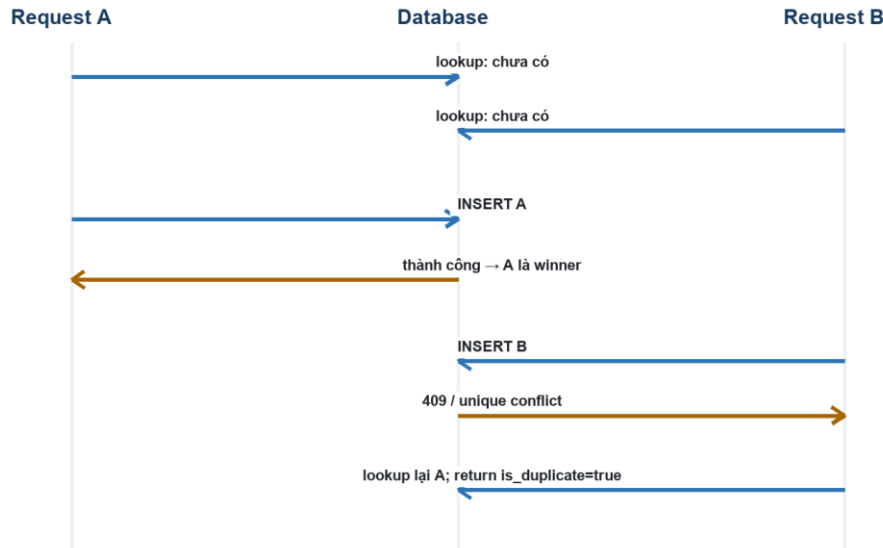
2.3. Kỹ thuật 2 — partial unique index và retry winner

2.3.1. Vì sao application pre-check chưa đủ?

Lookup rồi insert là hai thao tác tách biệt. Hai request có thể cùng lookup trước khi bất kỳ request nào insert. Nếu chỉ dựa vào code application, cả hai sẽ tạo metadata và object trùng. Vì vậy correctness phải được khóa bằng unique invariant trong database.

Hai upload đồng thời: database chọn canonical winner

Pre-check giúp nhanh; partial unique index mới bảo đảm correctness



Hình 3 — Pre-check tối ưu latency; partial unique index quyết định winner khi có race.

2.3.2. Partial unique nghĩa là gì?

Index unique áp dụng lên (owner_id, notebook_id, content_hash) nhưng chỉ cho tập row thỏa predicate: active, không failed và canonical_document_id IS NULL. Gọi là partial vì row archived/failed/alias nằm ngoài invariant canonical active. Nhờ đó lịch sử và alias vẫn có thể tồn tại mà không phá khóa canonical.

```

UNIQUE (owner_id, notebook_id, content_hash)
WHERE content_hash IS NOT NULL
  AND is_active = true
  AND status <> 'failed'
  AND canonical_document_id IS NULL
  
```

2.3.3. Retry winner trong service

PostgREST adapter ánh xạ HTTP 409/unique violation thành DocumentDuplicateError. Service bắt lỗi, lookup lại đúng owner/notebook/hash. Nếu thấy row A vừa commit, B trả A với is_duplicate=true. Nếu vẫn không thấy, service trả DOCUMENT_METADATA_CONFLICT thay vì tự đoán.

Thành phần	Vai trò	Nếu thiếu
Pre-check	Đường nhanh, thường tránh insert và exception	Hệ thống vẫn đúng nhưng latency/rate exception cao hơn
Partial unique index	Invariant cuối cùng chống race	Có thể sinh hai canonical active cùng hash
Retry lookup winner	Biến unique conflict thành response idempotent thân thiện	Request B nhận lỗi dù hệ thống đã có document đúng

Câu nói nên dùng khi bảo vệ

Pre-check tối ưu latency; unique index bảo đảm correctness. Database là nơi quyết định canonical winner. Request thua race không retry insert, mà lookup winner và trả is_duplicate=true.

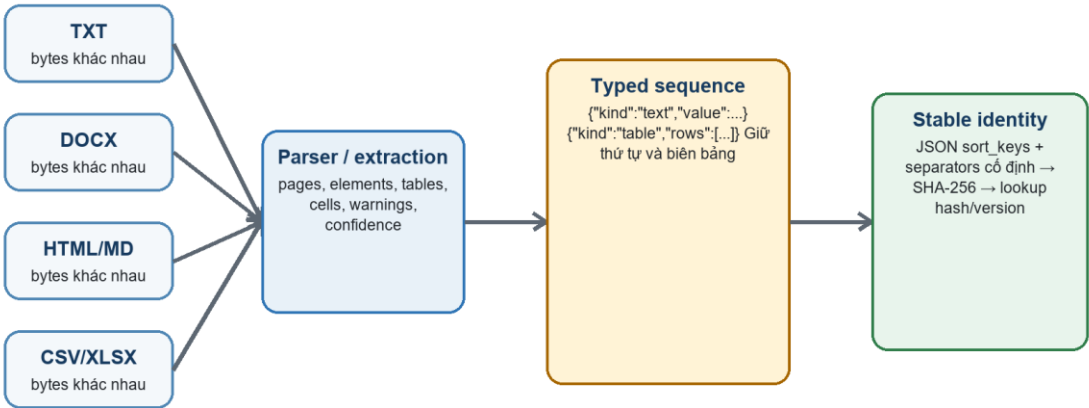
2.4. Kỹ thuật 3 — typed canonical projection cho duplicate khác định dạng

2.4.1. Vì sao raw SHA-256 không đủ?

Cùng một bảng lưu CSV và XLSX có ZIP/XML, metadata và bytes hoàn toàn khác. Một đoạn văn lưu TXT, DOCX, Markdown hoặc HTML cũng có bytes khác. Raw hash bắt buộc khác dù người đọc thấy cùng nội dung. Muốn so xuyên format, cần parse rồi đưa output parser về một representation trung lập, versioned và deterministic.

Typed canonical projection: so sánh ý nghĩa ổn định

Extraction tạo dữ liệu; projection chọn representation có thể so sánh xuyên định dạng



Không flatten bảng thành prose; không dùng NFKC/casfold làm identity authoritative; visual/OCR yếu sẽ chặn auto-alias.

Hình 4 — Extraction và typed canonical projection là hai bước bổ sung nhau.

2.4.2. Tại sao không chỉ dùng extraction?

Extraction trả dữ liệu giàu chi tiết nhưng còn phụ thuộc parser: page boundary, block IDs, vị trí, cú pháp Markdown, HTML markup, metadata DOCX và cách parser biểu diễn bảng. Nếu serialize toàn bộ extraction output, hai file cùng visible content vẫn khác vì các chi tiết phụ. Nếu chỉ lấy parsed.text rồi flatten, prose và table có thể va vào cùng chuỗi, đồng thời mất row/column order và table boundary. Canonical projection nằm giữa hai cực đó: bỏ chi tiết trình bày không ảnh hưởng semantics, nhưng giữ loại element và cấu trúc cần thiết cho exact identity.

Cách làm	Ưu điểm	Sai hỏng có thể xảy ra	Lựa chọn repo
Hash raw extraction JSON	Giữ mọi chi tiết	Quá nhạy với parser/format/page metadata	Không dùng làm cross-format exact
Flatten mọi thứ thành text	Đơn giản	Bảng và prose va identity; mất order/boundary	Bị chặn bằng kind typed
Typed canonical projection	Ổn định xuyên format nhưng bảo toàn semantics	Cần version/trust gate	Được dùng làm authoritative identity

2.4.3. Thuật toán projection từng bước

1. Khởi tạo sequence rỗng; map table_id → ParsedTable; identity_trusted = not parsed.ocr_used; đếm visual chưa biểu diễn.
2. Duyệt pages theo thứ tự; nếu page có elements thì duyệt elements theo thứ tự parser cung cấp.
3. Element text: với Markdown, bỏ syntax trình bày/đếm visual; strict_normalize_text; gộp text liên tiếp thành {kind:'text', value:...}.
4. Element table có ParsedTable tương ứng: canonicalize từng cell rồi thêm {kind:'table', rows:[...], ...}; giữ table boundary, row order và column order.
5. Element khai báo table nhưng không chứng minh được cell boundaries: giữ {kind:'unstructured_table', value:...} và hạ identity_trusted=false.
6. Nếu parser không có element, fallback page_text/document_text; nếu table không có vị trí tương đối với prose, vẫn đưa vào signature nhưng chặn auto-alias.
7. Hạ trust khi parser confidence < 0.9, có replacement warning, OCR hoặc visual chưa biểu diễn.
8. Serialize {'profile': normalization_version, 'sequence': sequence} bằng JSON ensure_ascii=false, sort_keys=true và separators cố định.
9. Tính H_canonical = SHA256(canonical_payload); đồng thời giữ linear_text cho loose signals/template detection.

```
{
  "profile": "knowledge-document-identity-v2",
  "sequence": [
    {"kind": "text", "value": "Bảng giá tháng 8"},
    {"kind": "table", "rows": [[ "Mã cần", "Giá"], [ "A101", "4.5 tỷ" ] ]}
  ]
}
H_canonical = SHA256(stable_json_above)
```

2.4.4. Vì sao cần kind=text và kind=table?

Giả sử prose ghi 'A101 | 4.5 tỷ' và một bảng có hai cell A101, 4.5 tỷ. Nếu flatten, hai chuỗi có thể giống. Typed projection bắt buộc một bên là kind=text, bên kia là kind=table, vì vậy JSON và hash khác. Đây là bảo vệ semantics: hình thức cấu trúc là một phần của exact identity.

2.4.5. Vì sao giữ row/column order?

Đổi thứ tự row hoặc column có thể đổi ý nghĩa quan hệ giữa header và value, hoặc thứ tự ưu tiên nghiệp vụ. Document exact identity phải bảo thủ: order khác → hash khác. Bài toán muốn ghép row không phụ thuộc vị trí được giải riêng ở structured table diff bằng business row identity, không nên nói lỏng document exact hash.

2.4.6. Strict normalization và loose normalization

Profile	Thao tác	Được dùng để auto-merge?	Lý do
Strict	Unicode NFC, NBSP/khoảng trắng ổn định	Có, nếu qua trust gate	Giữ phân biệt semantics, không casefold/NFKC quá mạnh
Loose	Unicode NFKC, casefold, tokenization/signature	Không	Chỉ tìm candidate; có thể gom các biểu diễn khác nghĩa

Ví dụ, NFKC có thể làm một số compatibility characters gần nhau hơn. Điều đó hữu ích khi tìm near candidate nhưng quá rủi ro để khẳng định exact identity. Repo ghi rõ loose normalization không được dùng làm auto-merge.

2.4.7. Trust gate: điều kiện nào mới được auto-alias?

Guard	Điều kiện đạt	Nếu không đạt
Parser representation	Cell boundary/table location chứng minh được	Chỉ candidate; không auto identity
OCR	parsed.ocr_used = false	identity_trusted=false
Visual	Không còn image/visual chưa biểu diễn	identity_trusted=false
Parser confidence	Không thấp hơn 0.9	identity_trusted=false

Guard	Điều kiện đạt	Nếu không đạt
Encoding quality	Không có replacement-character warning	identity_trusted=false
Độ dài	Ít nhất 40 ký tự và 6 token	Không auto-alias tài liệu quá ngắn

Ý nghĩa của trust gate

Hai hash canonical giống nhau chưa đủ. Hệ thống còn yêu cầu parser chứng minh representation đáng tin và nội dung đủ dài. Đây là biện pháp chống false positive khi extraction mất hình, mất table boundary hoặc text quá ngắn.

2.4.8. So sánh với tài liệu cũ và dữ liệu được lưu ở đâu

1. File mới đã được upload Storage và có documents row, vì lớp raw bytes trước đó không match.
2. Worker download, verify raw hash, parse và tạo H_canonical/version trong RAM.
3. Nếu trust gate đạt, repository lookup document cũ bằng owner + notebook + normalized_content_hash + normalization_version, với ready/active/canonical filters.
4. Không cần download/parse lại file cũ: hash/version đã persist trong documents từ lần ingest trước.
5. Nếu mode=on và match, worker gọi atomic complete_duplicate_ingestion_job; database advisory-lock identity, reload canonical rồi chuyển document mới thành alias.
6. Alias row giữ object path và lịch sử; status=ready, canonical_document_id trở winner, is_current=false, quality_status=duplicate. Không tạo chunk/embedding/structured facts cho alias.
7. Nếu mode=shadow, chỉ ghi exact_content relation/observation; không suppress để quan sát an toàn trước rollout.

Điểm cần nói chính xác: cross-format duplicate không tiết kiệm upload object và parse, vì chỉ có thể biết sau extraction. Nó tiết kiệm contextual LLM enrichment, chunking/embedding và derived inventory phía sau.

2.4.9. Race ở normalized identity

Hai worker có thể cùng parse hai format khác nhau và cùng lookup 'chưa có canonical'. RPC completion không tin kết quả lookup cũ: nó lấy pg_advisory_xact_lock theo owner:notebook:normalization-version:hash, reload canonical sau lock và dùng conditional unique index làm invariant cuối. Worker đến sau chuyển sang duplicate_suppressed. Job claim_token/lease được kiểm tra trong cùng transaction để stale worker không commit.

2.5. Duplicate theo mức chunk trước embedding

2.5.1. Vì sao cần chunk-level dedup?

Hai document không exact vẫn có thể chia sẻ điều khoản, bảng con hoặc đoạn boilerplate. Nếu chỉ dedup document, các chunk giống hệt vẫn bị embed lại và chiếm vector inventory. Repo lập probe cho mọi chunk sau contextualization nhưng trước provider call.

2.5.2. Probe có hai checksum khác mục đích

Trường	Tính trên	Mục đích
strict_hash	canonical_content hoặc source chunk text sau strict normalization	Xác định exact content của chunk
loose_signature / SimHash	Canonical text đã tokenize/shingle	Tìm fuzzy candidate nhanh
embedding_text_checksum	Đúng chuỗi sẽ gửi embedding provider, có thể đã contextualize	Chỉ reuse vector khi input embedding thật sự giống
normalization_version	Profile identity	Không so/reuse giữa hai thuật toán version khác
scope	Owner/notebook/document claim scope	Chặn cross-tenant/cross-scope candidate sai

Tại sao strict_hash giống vẫn chưa chắc reuse vector?

Chunk source giống nhưng contextual prefix/section khác thì embedding input khác. Repo bắt embedding_text_checksum giống, model giống và vector tồn tại. Nếu thiếu một điều kiện, vẫn có thể ghi exact relation nhưng provider phải embed lại.

2.5.3. SimHash được tính như thế nào?

SimHash biến tập đặc trưng của văn bản thành chữ ký 64 bit sao cho hai văn bản chia sẻ nhiều shingle thường có Hamming distance nhỏ. Trong repo, đặc trưng là 3-token shingles. Mỗi shingle được SHA-256; 64 bit đầu được dùng để cộng/trừ một vector trọng số 64 chiều. Bit cuối cùng bằng 1 khi tổng vị trí đó không âm, ngược lại bằng 0.

```
tokens = tokenize(text)
features = all 3-token shingles(tokens)
v[0..63] = 0
for feature in features:
    h = first_64_bits(SHA256(feature))
    for bit i in 0..63:
        v[i] += +1 if h[i] == 1 else -1
simhash[i] = 1 if v[i] >= 0 else 0
```

Khoảng cách Hamming được tính bằng `popcount(sigA XOR sigB)`: XOR đánh dấu các bit khác nhau; `bit_count` đếm số bit 1. Repo dùng ngưỡng mặc định 24 bit cho fuzzy probe, nhưng ngưỡng này chỉ lọc candidate.

2.5.4. LSH 8×8 giúp giảm tìm kiếm như thế nào?

Quét Hamming với toàn bộ chunk là tốn $O(N)$. Repo chia chữ ký 64 bit thành 8 band, mỗi band 8 bit và tạo tám expression index trong migration 10. RPC chỉ lấy chunk có strict hash exact hoặc trùng ít nhất một band. Mỗi probe tối đa 50 candidate, mỗi RPC tối đa 128 probes; exact được xếp trước, sau đó số band match.

```
64-bit SimHash = [band0][band1]...[band7]    # mỗi band 8 bit
candidate nếu strict_hash giống
             hoặc tồn tại i: band_i(new) == band_i(old)
sau đó application mới kiểm tra Hamming và full text relation
```

LSH không quyết định duplicate

Trùng một band có false positive. SQL chỉ sinh candidate. Application bắt `normalization_version`, kiểm tra Hamming, chạy `analyze_text_relation` trên full normalized text và fail closed nếu hash/text bất nhất.

2.5.5. Exact vector reuse và same-batch reuse

```
can_reuse = (candidate.embedding_model == current_model)
             and candidate.vector is not empty
             and candidate.embedding_text_checksum == probe.embedding_text_checksum
             and quality_mode == 'on'
```

Với database candidate, exact match ưu tiên candidate có vector reusable rồi sắp deterministic theo document/chunk ID. Trong cùng batch, chunks được group theo (`normalization_version`, `strict_hash`); text và embedding checksum phải giống, chunk sau ghi `reuse_from_chunk_index` trở representative. Pipeline.embed loại precomputed/dependency indexes khỏi danh sách `provider_texts`, gọi provider chỉ cho phần còn lại rồi điền vector đại diện.

Tình huống	Reuse vector?	Lý do
Strict content + cùng embedding input + cùng model	Có khi mode=on	Vector biểu diễn đúng cùng chuỗi/model
Strict content nhưng context prefix khác	Không	Embedding input checksum khác
Strict content nhưng model khác	Không	Không trộn embedding space
Near duplicate	Không	Fuzzy text không chứng minh vector tương đương

Tình huống	Reuse vector?	Lý do
Mode=shadow	Không	Chỉ quan sát decision, không thay hành vi
Cùng batch strict exact	Có	Representative được embed một lần; dependency lấy lại cùng vector

2.6. Near duplicate và document-level relation

Near duplicate có hai đường sinh candidate: SimHash-LSH trước embedding và ANN vector search sau embedding. ANN detector mặc định probe tối đa 8 chunks và top candidates có scope; sau đó aggregate theo target document, yêu cầu coverage tối thiểu cho near/version. Candidate cuối không dựa riêng cosine score mà chạy analyze_text_relation.

Signal trong analyze_text_relation	Nó giúp phân biệt gì?
Strict equality	Exact normalized text
3-shingle Jaccard + SequenceMatcher	Overlap từ/cấu trúc bề mặt
Token containment + line overlap	Một bản chứa phần lớn bản kia
Template/projection signature	Cùng mẫu nhưng value/scope khác
Quantity + unit	4.5 tỷ khác 4.8 tỷ; 70 m ² khác 70 USD
Date/effective cues	Bản cập nhật theo thời gian
Negation/modality	'được phép' khác 'không được phép'/'phải'
Extracted claim conflict	Value khác trên claim align được
Business scope	Khác tòa/căn/phạm vi thành variant/distinct thay vì conflict sai
Semantic score	Tăng evidence candidate nhưng không override critical difference

Nếu có critical difference đã xác thực, classifier ưu tiên conflict/template variant trước near duplicate. Nếu fuzzy match được persist, relation ở trạng thái pending và suppression_applied=false. Không auto-delete document và không đưa vector candidate vào precomputed_vectors.

2.7. MMR và collapse duplicate khi retrieval

Ingestion dedup giảm inventory, nhưng retrieval vẫn cần chống context lặp do legacy data hoặc nhiều version. MMR reranker trước hết collapse exact group/checksum, rồi chọn greedily theo công thức cân bằng relevance và redundancy.

```
MMR(c) = λ × relevance(c)
        - (1 - λ) × max_similarity(c, selected)
repo mặc định λ = 0.7; diversity dùng Jaccard trên 3-word shingles
```

MMR không sửa canonical inventory và không thay duplicate classifier. Nó chỉ quyết định context nào đưa vào prompt để tránh nhiều chunk giống nhau chiếm hết token budget.

2.8. Ma trận tình huống duplicate thường gặp

Tình huống	Lớp phát hiện	Kết quả	Lưu/không lưu
Cùng file, đổi filename	Raw SHA-256	Return canonical is_duplicate=true	Không object/document/job mới
Cùng bytes, hai upload đồng thời	DB partial unique	Một winner; loser lookup lại winner	Loser không upload object
TXT và DOCX cùng prose	Canonical typed projection	Auto-alias nếu trust gate và mode=on	Object/alias row giữ; không chunks/vectors mới
CSV và XLSX cùng bảng	Canonical table projection	Auto-alias nếu rows/cells/order giống	Như trên
Bảng đổi row order	Canonical hash khác	Không exact document	Đi tiếp chunk/structured diff
Bảng flatten thành prose	kind khác	Không exact	Giữ hai document
OCR/visual chưa biểu diễn	Trust gate	Không auto-alias dù signature candidate	Giữ và review
Document khác nhưng có đoạn boilerplate exact	Chunk strict hash	Có thể reuse vector exact chunk	Document vẫn riêng; chunk metadata ghi decision
Chunk source giống nhưng context khác	Embedding checksum guard	Không reuse vector	Embed riêng
Paraphrase gần giống	LSH/ANN + analyzer	Near duplicate pending	Giữ cả hai, không suppression/reuse
Câu giống nhưng số/negation khác	Claim/critical difference	Conflict/variant thay vì near	Giữ cả hai
Cùng file ở tenant/notebook khác	Scope filter	Không bị coi duplicate cross-scope	Mỗi scope có canonical riêng

2.9. Chế độ rollout: off, shadow, on

Mode	Identity/candidate	Có suppression/reuse?	Mục đích
off	Bỏ quality path tương ứng	Không	Kill switch
shadow	Tính/ghi observation hoặc relation	Không	Đo false positive trước rollout
on	Chạy full path	Chỉ exact + đủ guard	Tiết kiệm chi phí có kiểm soát

RPC duplicate còn kiểm tra cả mode được enqueue trong job và effective runtime mode. Worker/job cũ không được tự động suppress nếu hệ thống đã hạ mode vì sự cố.

2.10. Dẫn chứng kiểm thử duplicate

Test	Điều được chứng minh
test_service_skips_upload_when_exact_hash_duplicate_exists	Exact pre-check trả document cũ; storage.uploads rỗng.
test_service_resolves_concurrent_exact_upload_from_unique_constraint	Unique conflict được lookup winner, không upload object.
test_prose_identity_matches_across_txt_docx_markdown_and_html	Raw hash khác, strict canonical hash giống.
test_structured_table_identity_matches_across_csv_xlsx_docx_and_html	Typed table identity xuyên bốn format.
test_table_value_or_order_change_does_not_match	Value/order đổi làm exact identity đổi.
test_flat_table_text_is_not_auto_equated_with_proven_structured_cells	kind table chặn flatten collision.
test_worker_auto_aliases_strict_content_duplicate_without_embedding	Exact canonical dừng trước contextual LLM/vector.
test_exact_chunk_reuses_compatible_persisted_embedding	Cùng model/input exact nhận vector persisted.
test_exact_content_with_different_embedding_context_is_not_reused	Context checksum guard hoạt động.
test_same_batch_exact_chunks_share_the_representative_vector	Dependency reuse trong batch.
test_pipeline_embeds_only_chunks_without_exact_reuse_vectors	Embedding provider chỉ nhận phần chưa reuse.
test_near_duplicate_is_flagged_but_not_reused	Fuzzy relation không tạo precomputed vector.
test_number_change_is_a_conflict_candidate_and_keeps_both_vectors	Khác số không bị near-merge sai.

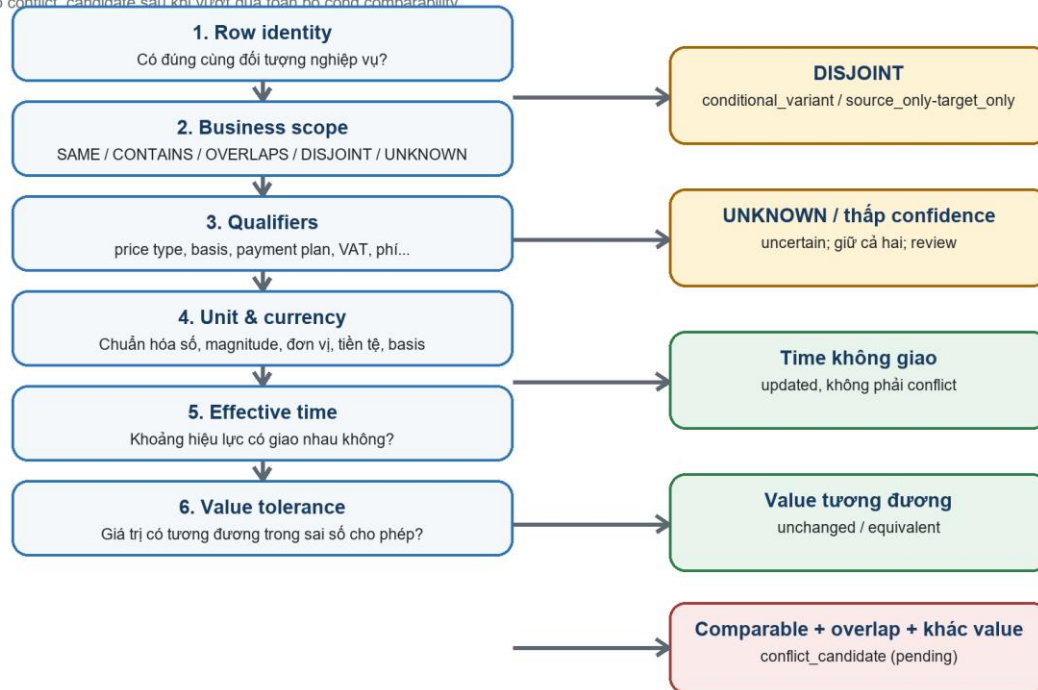
Kết quả audit duplicate
Nhóm test duplicate trực tiếp: 99 passed. Nhóm adapter/integration/evaluation bổ sung: 33 passed. Checked-in benchmark ghi exact auto-reuse TP=5, TN=24, FP=0, FN=0, recall=1.0. Đây là local/static evidence, chưa phải production race proof.

3. Conflict — từ prose đến structured facts

Conflict là bài toán chứng minh hai phát biểu đang nói về cùng đối tượng, cùng điều kiện và cùng khoảng hiệu lực nhưng khác giá trị. Chỉ nhìn thấy hai con số khác nhau là chưa đủ. Repo có hai đường bổ sung nhau: generic claim analysis cho prose/chunk và structured-fact subsystem cho bảng nghiệp vụ lớn.

Conflict không phải là ‘hai con số khác nhau’

Chỉ tạo `conflict_candidate` sau khi vượt qua toàn bộ cổng comparability



Hình 5 — Chuỗi cổng comparability trước khi tạo `conflict_candidate`.

3.1. Định nghĩa quyết định trong hệ thống

Kết quả	Điều kiện rút gọn	Có cần review?	Có xóa/gộp dữ liệu?
unchanged/equivalent	Comparable và value tương đương	Không; auto_confirmed	Không xóa; có relation
updated	Cùng claim nhưng effective intervals không chồng lấn	Thường không; deterministic	Giữ cả hai theo lịch sử
conditional_variant	Scope hoặc qualifier disjoint	Thường không	Giữ cả hai theo điều kiện
conflict_candidate	Comparable + time overlap + value mismatch	Có; pending	Giữ cả hai
uncertain	Thiếu/không tin cậy scope, time, qualifier, identity	Có; pending	Giữ mọi evidence
added/removed	Chỉ có ở một snapshot sau join	Tùy policy	Không suy diễn là conflict

Kết quả	Điều kiện rút gọn	Có cần review?	Có xóa/gộp dữ liệu?
source_only/target_only	Không có cặp claim/bảng comparable	Tùy policy	Giữ độc lập

3.2. Đường generic text claim hoạt động như thế nào?

Khi detector đánh giá hai đoạn/document gần nhau, extract_claims tìm claim từ prose rồi align bằng claim key/similarity. Nó chuẩn hóa quantity bằng Decimal, magnitude và unit; nhận diện date, negation và policy modality. detect_claim_conflicts chỉ báo khác biệt khi hai claim align đủ mạnh và một signal có ý nghĩa thay đổi.

Signal	Ví dụ	Cách tránh false conflict
Quantity	4,5 tỷ ↔ 4.500.000.000	Normalize Decimal × magnitude; cùng unit có thể equivalent
Unit	70 m ² ↔ 70 ft ²	Không so value trực tiếp nếu unit không tương thích
Date	áp dụng 01/08 ↔ 01/09	Phân biệt update/time difference
Negation	được phép ↔ không được phép	Critical difference, không near-merge
Modality	có thể ↔ phải	Policy modality difference
Scope	tòa S1 ↔ tòa S2	Template variant/distinct thay vì cùng claim

Đường generic phù hợp câu văn và đoạn ngắn, nhưng post-embedding document detector chỉ dùng số probe giới hạn cho candidate coverage. Nó không được dùng để khẳng định đã diff mọi row của bảng 1.000 dòng. Bảng lớn đi qua subsystem structured facts.

3.3. Vì sao bảng cần structured facts riêng?

Một chunk text của bảng có thể mất header context, cắt giữa row hoặc sampling chỉ thấy vài hàng. Vector similarity cho biết hai đoạn có chủ đề gần nhau, không chứng minh row A101 trong tòa S1 và cùng loại giá/hiệu lực. Structured analyzer tận dụng ParsedTable đã có từ extraction, đi qua toàn bộ data_rows và tạo claim có khóa nghiệp vụ/provenance.

Không parse/lưu một bảng raw lần thứ hai

Worker dùng prepared.parsed_document.tables đã được parser tạo ở pipeline.prepare.

TableAnalysis và relation batch tồn tại trong RAM; persistence lưu schema hash, content hash, claim, source cells và provenance cần audit, không lưu thêm một raw table dump độc lập.

3.4. Mô hình một structured claim

```
StructuredClaim = {
  subject_key, predicate, normalized_value,
  business_scope: {location, product, commercial},
  qualifiers: {stable, optional},
  temporal: {publication, effective_from, effective_to, observed, ingested},
  authority: {source_type, publisher, approval, officiality, level},
  provenance: {table_id, row_index, data_row_ordinal, cell, page, source_chunk_id},
  extraction_confidence, extractor_version
}
```

Nhóm trường	Tại sao cần
subject_key + predicate	Xác định 'đối tượng nào, thuộc tính nào' để join claim
normalized_value	So giá trị sau chuẩn hóa number/money/area thay vì raw spelling
scope	Không ghép A101/S1 với A101/S2 hoặc retail với wholesale
qualifiers	Không coi giá niêm yết và giá sau chiết khấu là conflict
temporal	Không coi giá tháng 7 và tháng 8 là conflict nếu hiệu lực nối tiếp
authority	Đưa bằng chứng nguồn cho review/retrieval; không tự chọn winner
provenance	Dẫn lại row/cell/chunk/page thật, kiểm tra citation
confidence/version	Hạ uncertain khi extraction yếu và tái lập detector

3.5. Analyzer xử lý toàn bộ bảng và tạo row identity

3.5.1. Luồng analyze_table

1. Tách header row và data rows; chuẩn hóa semantic header, phát hiện duplicate semantic columns.
2. Duyệt mọi data row, không sampling. Test 1.000 row tạo đủ 1.000 claims.
3. Tạo row identity theo location/product fields, không dựa vào vị trí row.
4. Đọc qualifier và temporal ở row trước, document metadata sau; kiểm tra interval đảo.
5. Chuẩn hóa money/area/number, currency, unit, magnitude và price basis.
6. Tạo provenance chính xác: physical row_index, data_row_ordinal, source cells, table/page/block.
7. Confidence cuối lấy min của table/header/identity/value/cell/temporal; threshold trusted là 0.70.
8. Có thể tạo derived total price = price_per_area × area; so với published total bằng tolerance.

3.5.2. Row identity được dựng theo thứ tự nào?

_row_identity ưu tiên location/project/phase/subdivision/building/unit. Project một mình không đủ. Sau đó thêm id/code/name hoặc composite product attributes như property_type, bedrooms, floor, direction, layout, area. Nếu evidence vẫn thiếu, khóa fallback unresolved:{table}:row:{index} có confidence 0.35; nó không được dùng để tạo conflict chắc chắn.

Row	Identity đúng	Sai nếu chỉ dùng mã căn
Ocean Park, S1, A101	project=ocean park building=s1 unit=a101	A101 có thể tồn tại ở nhiều tòa
Ocean Park, S2, A101	project=ocean park building=s2 unit=a101	Sẽ bị ghép nhầm với S1
Không có unit; 2PN, tầng 15, Đông Nam, 70m ²	Composite product identity	Dùng row position sẽ hỏng khi sort
Không đủ khóa	unresolved:... + low confidence	Không được giả tạo identity chắc chắn

3.6. Business scope đa facet và quan hệ có hướng

BusinessScope gồm LocationScope, ProductScope và CommercialScope. document_type chỉ là routing metadata, không được dùng để tách identity nghiệp vụ. So sánh scope trả quan hệ có hướng để phân biệt broad/narrow. Missing field không được coi là wildcard vì điều đó dễ ghép sai; không có anchor chung sẽ là UNKNOWN.

Quan hệ	Ý nghĩa	Hệ quả conflict
SAME	Hai scope tương đương	Đi tiếp qualifier/time/value
LEFT_CONTAINS_RIGHT	Bên trái rộng hơn bên phải	Comparable theo policy nhưng cần giữ direction/evidence
RIGHT_CONTAINS_LEFT	Bên phải rộng hơn	Tương tự, hướng ngược lại
OVERLAPS	Có phần giao nhưng không chứa hoàn toàn	Có thể comparable; phải giải thích overlap
DISJOINT	Một facet rõ ràng khác nhau	conditional_variant hoặc source/target-only, không conflict
UNKNOWN	Thiếu anchor/evidence	uncertain, không giả định equal

Ví dụ: project=Ocean Park + building=S1 và project=Ocean Park + building=S2 là DISJOINT dù cùng project. Một scope project=Ocean Park và scope project=Ocean Park+building=S1 có quan hệ LEFT_CONTAINS_RIGHT, không phải SAME.

3.7. Qualifier giá: stable và optional

ClaimQualifiers tách stable qualifiers dùng trong candidate identity và optional qualifiers vẫn cần so trước conflict. Stable gồm price type, price basis, payment plan, discount program. Optional gồm VAT và maintenance fee. Việc optional không vào candidate key giúp hai claim vẫn được đưa cạnh nhau để kiểm tra; nó không có nghĩa optional bị bỏ qua.

Loại	Ví dụ	Tham gia candidate key?	Khi khác/thiếu
------	-------	-------------------------	----------------

Loại	Ví dụ	Tham gia candidate key?	Khi khác/thiếu
Stable	list price, price/m ² , standard plan, summer discount	Có	Khác rõ → conditional_variant; thiếu → uncertain
Optional	VAT included, maintenance fee included	Không	Vẫn so field-by-field; missing một phía → UNKNOWN/uncertain

Ví dụ cần trình bày

Giá niêm yết 4,5 tỷ và giá sau chiết khấu 4,3 tỷ không phải conflict nếu qualifier price_type/discount_program khác. Giá 4,5 tỷ 'đã gồm VAT' và 4,5 tỷ không nói VAT cũng không được coi chắc chắn bằng nhau; kết quả an toàn là uncertain.

3.8. Temporal: thời điểm xuất bản không phải thời gian hiệu lực

TemporalContext giữ riêng publication_time, effective_from/to, observed_at và ingested_at. Publication/ingestion chỉ nói khi tài liệu xuất hiện trong hệ thống; không chứng minh giá có hiệu lực từ lúc đó. compare_temporal_intervals chỉ dùng effective interval cho quan hệ update/conflict.

Khoảng A và B	Kết luận	Ví dụ
Không giao nhau / nối tiếp	updated	Giá tháng 7 và giá tháng 8
Giao nhau / cùng khoảng / contains	Có thể conflict nếu value khác	Hai bảng cùng áp dụng 01–31/08
Một/both thiếu effective interval	UNKNOWN → uncertain	Chỉ có ngày upload hoặc publication
effective_from > effective_to	Invalid, giảm confidence/cảnh báo	Row nhập ngược mốc thời gian

3.9. Chuẩn hóa value, currency, unit, basis và tolerance

Value được parse thành kiểu money/area/number với Decimal, magnitude, currency và unit. So sánh chỉ đi tiếp khi unit, currency và basis tương thích. Giá 64,285 triệu/m² × 70 m² có thể tạo derived total 4.499.950.000 và được coi tương đương 4,5 tỷ nếu nằm trong tolerance. Tolerance không dùng để che khác biệt lớn; nó giải sai số làm tròn/derived.

```
equivalent(a, b) nếu |a - b| ≤ max(absolute_tolerance, relative_tolerance × scale)
chỉ áp dụng sau khi unit/currency/price-basis tương thích
```

Cặp giá trị	Kết quả hợp lý	Lý do
4,5 tỷ VND ↔ 4.500.000.000 VND	Equivalent	Magnitude được normalize
4.499.950.000 ↔ 4.500.000.000 VND	Equivalent nếu tolerance	Sai số derived/làm tròn nhỏ
4,5 tỷ total ↔ 64,285 triệu/m ²	Không so trực tiếp	Price basis khác; cần area/derivation
4,5 tỷ VND ↔ 4,5 triệu USD	Variant/uncertain	Currency khác, không có conversion authority
4,5 ↔ 4,8 tỷ cùng mọi điều kiện	Conflict candidate nếu time overlap	Khác vượt tolerance

3.10. Source authority được dùng đúng chỗ

SourceAuthority thu thập source_type, publisher, approval, officiality, authority_level và metadata. Nó được gắn vào claim, lưu trong snapshot/claim và đưa ra retrieval/review. Tuy nhiên authority không tham gia candidate identity và không tự động làm official source thắng. Trước hết phải chứng minh hai claim comparable; sau đó authority chỉ là evidence để con người/policy chọn nguồn ưu tiên.

Vì sao không cho authority quyết định từ đầu?

Nguồn chính thức về tòa S1 không thể 'thắng' một nguồn môi giới về tòa S2 vì hai claim không cùng scope. Authority chỉ có ý nghĩa sau comparability, nếu không sẽ che lấp ghép identity.

3.11. Ghép bảng cũ và bảng mới

Worker load prior candidates từ structured store trước khi replace current facts. comparison.py rehydrate prior claims từ payload DB. Bảng được match bằng semantic schema fingerprint và overlap candidate identities; raw header aliases vẫn lưu audit nhưng không định nghĩa table family. Chỉ unique best match được dùng. Ambiguity trong một prior document không bị đoán; current table vẫn có thể so độc lập với nhiều prior documents.

Identity	Tính từ	Mục đích
input_content_hash	Canonical table input gồm content	Phát hiện nội dung bảng thay đổi
schema_fingerprint	Canonical semantic columns	Ghép cùng table family xuyên header alias/ngôn ngữ
row_identity_hash	subject_key nghiệp vụ	Join row không phụ thuộc vị trí
candidate_identity_hash	subject + predicate + stable qualifier	Gom claims có khả năng comparable
qualifier_hash	Stable qualifier payload	Join nhanh trong candidate group
claim_identity_hash	Full claim identity	Audit/dedup deterministic

3.12. Full-table diff $O(n+m)$

diff_table_analyses không tạo cross-product mọi row. Nó group claims hai phía vào hash map theo (subject_key, predicate), sau đó group theo stable qualifier. Xây map trái tốn $O(n)$, map phải $O(m)$; duyệt union keys và so cặp tương ứng gần $O(n+m)$ trong trường hợp key hợp lệ. Memory $O(n+m)$.

```
left_map [(subject_key, predicate)][stable_qualifier] -> claim(s)
right_map[(subject_key, predicate)][stable_qualifier] -> claim(s)
for key in union(left_map, right_map):
    compare matching qualifier groups
    one-sided groups -> added/removed/source_only/target_only
```

Nếu một business key có nhiều claim mơ hồ, code không zip theo row position.

_ambiguous_duplicate_relations giữ từng claim một phía với uncertain/reason=duplicate_business_key.

Điều này tránh ghép cặp giả khi bảng có duplicate key.

Bảng chứng quy mô

Test full-table có 1.000 row cũ, row mới bị đảo thứ tự, hai giá đổi, một row thêm và một row mất. Kết quả: 997 unchanged, 2 conflict, 1 added, 1 removed — tổng 1.001 relations. Điều này chứng minh join theo identity, không theo vị trí.

3.13. Decision pipeline chính xác của một cặp claim

```
compare(left, right):
    if low_confidence:
        return uncertain
    if scope == DISJOINT:
        return conditional_variant
    if scope == UNKNOWN:
        return uncertain
    if qualifier == DISJOINT:
        return conditional_variant
    if qualifier == UNKNOWN:
        return uncertain
    if unit/currency/basis incompatible:
        return variant_or_uncertain
    if values_equivalent_with_tolerance:
        return unchanged
    if effective_intervals_do_not_overlap:
        return updated
    if effective_time_unknown:
        return uncertain
    return conflict_candidate      # comparable + overlap + mismatch
```

Thứ tự guard là một phần của correctness. Ví dụ value phải được kiểm tra tương đương trước conflict; time phải được xét sau comparability nhưng trước conflict; authority không nằm trong gate comparability.

3.14. Ma trận các trường hợp conflict/variant/update

Trường hợp	Kết quả	Giải thích
------------	---------	------------

Trường hợp	Kết quả	Giải thích
Cùng căn, loại giá, time; value bằng sau normalize	unchanged	Cùng claim và equivalent
Cùng căn, value khác; effective months nối tiếp	updated	Không overlap nên là phiên bản theo thời gian
Cùng căn, value khác; cùng effective interval	conflict_candidate	Đã comparable và value mismatch
A101 tòa S1 ↔ A101 tòa S2	conditional_variant/source-only	Business scope disjoint
Cùng căn nhưng list price ↔ discounted price	conditional_variant	Stable qualifier khác
Một bên nói VAT, bên kia không nói	uncertain	Optional qualifier missing không được giả equal
Một bên thiếu effective time	uncertain	Publication/ingestion không suy ra validity
Currency khác	variant/uncertain	Không tự conversion
Giá/m ² × diện tích gần bằng total price	unchanged/equivalent	Derived value nằm trong tolerance
Cùng key xuất hiện hai lần mỗi bảng	uncertain một phía	Không zip theo vị trí; duplicate_business_key
Row mới không có ở prior	added/source_only	Không tự gọi conflict
Row cũ biến mất	removed/target_only	Không tự suy ra xóa nghiệp vụ
Extraction confidence < 0.70	uncertain	Không đủ trust cho quyết định value
Nguồn official 90 ↔ broker 20, claim comparable	Vẫn giữ relation; authority evidence	Không tự chọn winner trong detector

3.15. Persistence atomic, tenant scope và concurrency

build_structured_fact_persistence_batch tạo deterministic payload. source_chunk_id chỉ được gán khi chứng minh table source block và row range nằm trong chunk; không có nearest-chunk fallback. Atomic RPC replace_structured_facts_for_document khóa job/document FOR UPDATE, lấy advisory transaction lock, xóa/thay đúng document+extractor_version và rollback toàn transaction nếu lỗi.

Bảng/record	Dữ liệu chính	Tính audit/an toàn
table_snapshots	Schema/input hash, page/chunk locator, temporal/authority, confidence/warnings	Một version snapshot có provenance
structured_claims	Row/candidate/claim hashes, normalized value, qualifier, time, source cells/chunk	Composite tenant FKs và citable evidence
claim_relations	Hai snapshot/claim, relation type, confidence, reason, detector, review status	Pending/auto-confirmed; unique detector key
audit log	Before/after/action/actor/reason/timestamp	Append-only; hỗ trợ review trace

Worker chỉ persist structured facts sau khi document completion disposition=completed; duplicate_suppressed không có facts riêng. Sau write, worker reload candidates và cho phép tối đa một bounded reconciliation replace bổ sung để khép race hai document ingest đồng thời. Structured persistence failure fail-open đối với vector RAG: ingestion chính vẫn hoàn tất và structured path retry sau.

3.16. Retrieval-first, cảnh báo conflict và citation hai phía

ChatService parse structured intent và gọi search_structured_claims trước vector retrieval khi structured mode khác off. Query truyền notebook/document scope, predicate, subject, effective interval, qualifiers và limit. Chỉ claim có source_chunk_id mới được trả; subject match theo segment để A101 không khớp A1010; time query fail closed nếu claim thiếu effective interval.

1. Structured search chạy trước và trả claims kèm unresolved relation warnings.
2. Nếu mode=on và có evidence, chat dùng structured path; nếu lỗi/không có evidence thì fallback vector retrieval.
3. _structured_candidates dùng UUID chunk thật làm citation ID và giữ qualifier/time/authority/relation warning dạng object.
4. Generator nhóm hai nguồn theo cùng relation_id cho conflict/conflict_candidate; dismissed relation bị bỏ.
5. Prompt yêu cầu trình bày bất đồng và cite hai phía. Sau stream, code kiểm tra; nếu model cite thiếu một phía, nó nổi cảnh báo và phát CitationHit còn thiếu.

Điểm mạnh cần nhấn mạnh

Bắt buộc citation hai phía không chỉ dựa vào prompt. openai_generator.py enforce bằng relation_id sau khi model stream, nên conflict chưa giải quyết không bị trình bày như một sự thật đơn nguồn.

3.17. Human review và optimistic concurrency

API có list pending relations, tải evidence hai snapshot/claim và resolve. Reviewer gửi action, reason bắt buộc và expected_updated_at. RPC chỉ update nếu timestamp còn khớp; relation đã bị người khác sửa trả 409 để client refresh. Notebook ownership failure được ẩn thành 404. Review dùng user JWT; derived tables không cho authenticated user mutate trực tiếp.

Action	Ý nghĩa kết quả
confirm	Xác nhận relation hiện tại
confirm_equivalent	Xác nhận hai claim tương đương
confirm_update	Xác nhận quan hệ cập nhật theo thời gian

Action	Ý nghĩa kết quả
confirm_conflict	Xác nhận bất đồng thật
confirm_conditional_variant	Xác nhận khác điều kiện/scope
dismiss	Bác candidate; review_status=dismissed, không force conflict citation

3.18. Dẫn chứng kiểm thử conflict

Nhóm test	Các hành vi tiêu biểu
Scope	Different buildings disjoint; broad/narrow directional; overlap; empty scope unknown; commercial facets real constraints.
Qualifier	Stable candidate identity; optional missing unknown; case/order insensitive; price type variant; VAT missing uncertain.
Temporal	Publication không suy ra effective; invalid interval; sequential months update; unknown time blocks conflict.
Analyzer/row	Exact provenance; row currency; composite identity; no sampling 1.000 rows; derived total tolerance.
Table diff	Join by identity not position; duplicate key uncertain; comparable overlap price change pending conflict.
Persistence/SQL	Deterministic hashes; citable row guard; atomic service-only RPC; tenant composite FKs; RLS/grants.
Worker	Candidate load→diff→replace; reload after write closes race; runtime off kill switch; fail-open/retry.
Retrieval/generation	Structured search scope/time/qualifier; real chunk citation; both conflict citations enforced; dismissed not forced.
Review API	Pending pagination, evidence validation, optimistic timestamp, stale 409, blank reason/naive timestamp rejected.

Kết quả audit conflict

Scope/analyzer/diff/comparison/persistence/SQL contracts: 76 passed; domain/identity/authority: 17 passed; structured worker: 8 passed — tổng 101 test tập trung. Audit retrieval/review bổ sung: 84 passed. Không dùng các con số này để khẳng định live Supabase deployment.

4. Luồng tương tác và lưu tạm/lưu bền

Phần này mô tả hệ thống theo sequence thực thi, tập trung vào câu hỏi: so sánh gì với gì, dữ liệu hiện tại nằm ở đâu, khi nào ghi bền và khi có lỗi/race thì thành phần nào là authoritative.

4.1. Luồng A — upload lại cùng bytes, đổi tên file

Bước	Thành phần	Dữ liệu/so sánh	Trạng thái lưu trữ sau bước
A1	FastAPI/API	Đọc filename mới và bytes	Bytes trong RAM request
A2	Domain validate	Kiểm filename/type/size/signature; H_raw=SHA256(bytes)	ValidatedDocumentFile trong RAM
A3	Document repository	Lookup owner+notebook+H_raw với canonical filters	Đọc documents; chưa ghi gì
A4	DocumentService	Thấy canonical cũ, trả chính document đó	Không create row; không Storage object; không job
A5	Client	Nhận is_duplicate=true và ID cũ	Hệ thống không parse/embed lại

Điều filename làm và không làm

Filename vẫn được validate/sanitize để bảo vệ upload và hiển thị metadata. Nhưng nó không nằm trong H_raw, nên đổi ten.txt thành ban_sao.txt không tạo content identity mới.

4.2. Luồng B — hai request cùng bytes chạy đồng thời

Bước	Request A	Database	Request B
B1	Pre-check chưa thấy	Chưa có row	Pre-check chưa thấy
B2	INSERT uploading thành công	Unique key thuộc A	INSERT vi phạm partial unique
B3	Tiếp tục Storage/enqueue	Trả unique conflict cho B	Adapter ném DocumentDuplicateError
B4	—	Lookup key trả A	Service trả A, is_duplicate=true

Trong đường này B thua race trước object upload, nên không có orphan object của B. Nếu response insert/queue bị mất sau commit, service/repository có reconciliation/idempotent retry thay vì lập tức ghi side effect lần hai.

4.3. Luồng C — TXT và DOCX cùng visible content

Bước	So sánh/lựa chọn	RAM	Lưu bền
------	------------------	-----	---------

Bước	So sánh/lựa chọn	RAM	Lưu bền
C1	Raw hash khác → không match lớp 1	Bytes + H_raw	Tạo documents row, upload object, enqueue job
C2	Worker verify H_raw/size rồi parse	DocumentSource + ParsedDocument	Job running/lease/token
C3	Typed projection tạo H_canonical	Sequence/JSON/fingerprint	Chưa ghi canonical JSON như bảng tạm
C4	Lookup H_canonical/version thấy document cũ	Canonical ID	Đọc documents fingerprint
C5	RPC lock, reload và xác minh mode/token/lease	RPC payload	Alias row + exact relation + audit + job duplicate_suppressed
C6	Return trước contextualize/chunk/embed	Parsed objects được GC sau job	Object source vẫn giữ; không chunks/vectors/structured facts mới

Canonical JSON là intermediate deterministic trong worker. Các trường identity cần dùng lại được lưu dưới dạng normalized hash, normalization version, loose signature/quality metadata; lần sau so hash/version thay vì reparse file cũ.

4.4. Luồng D — document mới, có chunk exact cũ và bảng có conflict

1. Raw/canonical document identity không exact nên ingestion tiếp tục.
2. Pipeline contextualize chunks. Probe giữ canonical source identity và checksum của embedding input riêng.
3. RPC tìm exact/LSH candidates theo tenant; application verify full relation. Exact compatible lấy vector cũ, near candidate chỉ ghi relation.
4. Embedding provider chỉ nhận chunk chưa reuse; vectors mới/reused hợp thành EmbeddedChunk list trong RAM.
5. Nếu dùng external Qdrant, generation points được ghi với ingestion_generation=claim_token trước DB completion.
6. Worker renew lease; atomic completion RPC recheck claim_token/lease, persist document chunks/fingerprint/relations và quyết định completed hoặc duplicate_suppressed nếu race normalized identity xuất hiện.
7. Nếu completed, external generation được finalize bằng cách xóa generation cũ; nếu duplicate_suppressed, generation hiện tại bị xóa.
8. Structured analyzer đã tạo claims từ mọi row. Worker load prior claims, diff scope/qualifier/unit/time/value, build relation payload và atomic replace structured facts.
9. Conflict_candidate/uncertain được lưu pending; unchanged/update/variant deterministic có thể auto_confirmed. Không bên nào bị xóa.

4.5. Luồng E — người dùng hỏi về một số liệu đang conflict

Bước	Hành vi	Bảo vệ
E1 — Plan intent	Tách subject/predicate/qualifier/effective time và document scope	Không query ngoài notebook/user scope
E2 — Structured search	Query structured_claims có source_chunk_id; trả relation warnings	Time query fail closed; A101 không match A1010
E3 — Path selection	Có evidence + mode=on → structured; lỗi/rỗng → vector fallback	RAG vẫn hoạt động khi structured path lỗi
E4 — Candidate projection	Đưa value/scope/qualifier/time/authority và UUID chunk thật	Citation gắn provenance, không nearest-chunk giả
E5 — Generation	Nói rõ bất đồng; cite hai phía theo relation_id	Post-stream enforcement bổ sung citation thiếu
E6 — Telemetry	Ghi retrieval_path=structured vector	Phân tích được path thực tế

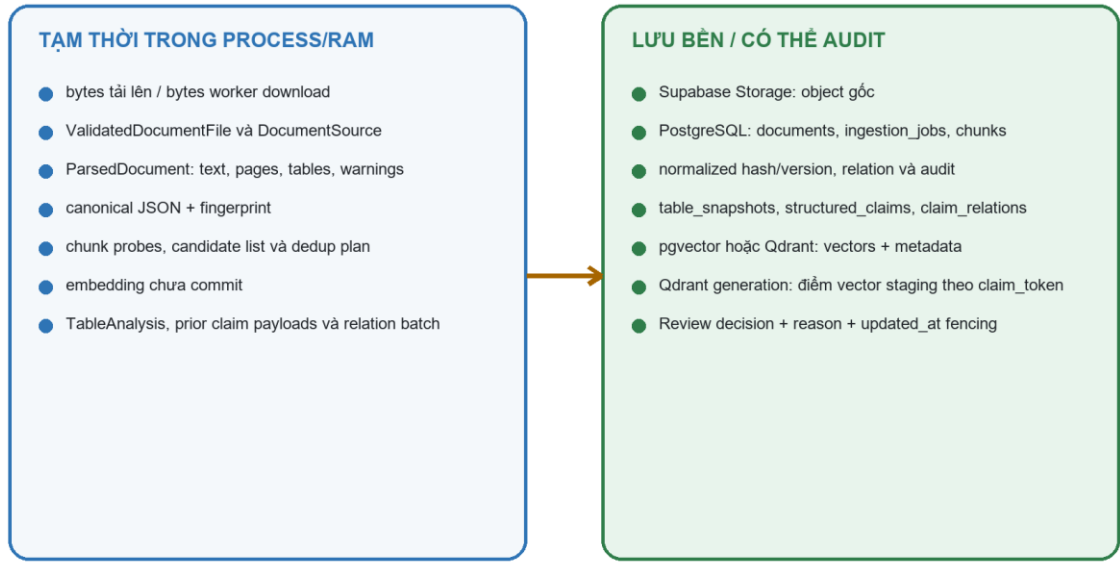
4.6. Luồng F — reviewer giải quyết conflict

1. Frontend list relation pending trong notebook; backend dùng user JWT và RLS/owner guard.
2. Reviewer mở evidence: repository tải hai snapshots và nullable claims; xác minh claim thuộc đúng snapshot.
3. UI hiển thị hai phía gồm normalized/raw cells, scope, qualifiers, effective time, authority, confidence và reason.
4. Reviewer chọn action và nhập reason; client gửi expected_updated_at của relation đã xem.
5. RPC so optimistic timestamp. Nếu đã thay đổi → 409 để refresh; nếu còn đúng → review_status confirmed/dismissed, resolved_by/at và audit được ghi atomic.
6. Dismissed relation không còn bắt generator cite như conflict; confirmed conflict tiếp tục force citation hai phía.

4.7. Bản đồ lưu tạm và lưu bền

Dữ liệu được giữ ở đâu trong lúc so sánh?

Không có bảng DB tạm dành riêng cho phép so sánh; có một ngoại lệ staging bền ở Qdrant generation



Hình 6 — Dữ liệu tạm trong process và dữ liệu lưu bền/audit.

Dữ liệu	Khi tạo	Nơi giữ	Khi nào mất/được thay
Upload bytes	API đọc UploadFile	RAM API	Mất sau request; object copy chỉ có nếu qua lớp exact
ValidatedDocumentFile	Sau validation/hash	RAM API	Mất sau service call
Object gốc	Sau create_uploading	Supabase Storage bucket documents	Giữ cho canonical/alias; upload fail thì best-effort delete
Document row	Trước object upload	PostgreSQL documents	Soft-delete giữ row; alias giữ canonical pointer
Job claim/lease/token	Enqueue/worker claim	PostgreSQL ingestion_jobs	Status/lease/token cập nhật transactional
Downloaded bytes	Worker download	RAM worker	Mất sau job; dùng verify raw hash
ParsedDocument	Pipeline.prepare	RAM worker	Mất sau job; parser nội bộ có thể có allocation riêng ngoài invariant audit
Canonical sequence/JSON	project_document_identity	RAM worker	Không có DB temp table; persist hash/version/metadata cần thiết
Chunk probes/candidates/plan	Trước embedding	RAM + RPC response	Decision metadata persist vào chunks/relations
Embedding vectors	Pipeline.embed	RAM trước commit	Sau đó pgvector/document_chunks hoặc external Qdrant

Dữ liệu	Khi tạo	Nơi giữ	Khi nào mất/được thay
Qdrant generation	External persist trước DB completion	Qdrant bền, key ingestion_generation	Delete nếu suppressed; finalize xóa old generation; reconcile nếu cleanup lỗi
TableAnalysis/prior claims	Structured analyzer/candidate load	RAM worker	Chuyển thành deterministic persistence batch
Snapshots/claims/relations	Atomic structured RPC	PostgreSQL	Replace đúng document+extractor version; review/audit giữ lịch sử quyết định

Không nên nói 'không có lưu tạm' một cách tuyệt đối

Ở tầng ứng dụng được audit, không có bảng DB/file cache tạm dành riêng cho comparison; các object trung gian ở RAM. Tuy nhiên thư viện parser có thể dùng buffer/temp nội bộ và Qdrant generation là staging bền. Vì vậy phát biểu đúng là 'không có temporary comparison table trong luồng application', không phải 'toàn hệ thống tuyệt đối không bao giờ dùng temp'.

4.8. Luồng lỗi và tính authoritative

Điểm lỗi	Cách xử lý	Nguồn authoritative
Storage upload fail/cancel	Mark document failed; best-effort delete object; trả lỗi	PostgreSQL status + object existence reconciled
Enqueue response mất	Retry idempotent/reconcile committed document	Database RPC state
Worker lease mất	_ensure_lease/renew fail; không đi tiếp completion	claim_token + lease trong DB
Chunk candidate lookup lỗi	Safe path tiếp tục không reuse	Correctness ưu tiên embed mới
External Qdrant đã ghi nhưng DB suppress	Delete generation theo document_id+claim_token	Fenced DB completion
Qdrant cleanup/finalize lỗi	Log và reconcile sau; không đổi DB success thành failure	Database completion authoritative
Structured candidate load lỗi	Vẫn replace current facts nếu có thể; không tạo relation cũ	Vector RAG completed; structured retry
Structured replace lỗi	Fail-open cho ingestion, log/retry	Canonical RAG inventory
Review stale	409, bắt refresh	claim_relations.updated_at

5. Cách chứng minh, giới hạn và deployment gate

5.1. Chứng minh đã làm thật: cần đưa ra bốn loại evidence

Loại evidence	Cần chỉ ra	Ví dụ trong repo
Đường gọi code	Entry point → decision → early return/persist	DocumentService.upload_file; IngestionWorker._process_job; ChatService._respond
Database invariant	Unique/check/FK/RLS/lock/RPC	Partial unique; advisory lock; claim_token lease; composite tenant FKs
Test hành vi	Assert side effect xảy ra/không xảy ra	storage.uploads==[]; provider chỉ nhận text chưa reuse; both citations enforced
Dữ liệu quan sát	Rows/relation/metadata/telemetry sau ca chạy	canonical_document_id; quality_status; chunk decision; claim_relations; retrieval_path

Chỉ trình chiếu một hàm SHA-256 chưa chứng minh duplicate end-to-end. Phải nối được hash vào lookup, unique invariant, early return, worker verify, persisted metadata và test side effects. Tương tự, conflict phải chứng minh analyzer xử lý mọi row, decision gate, atomic persistence, retrieval warning và review/citation.

5.2. Các lệnh test có thể chạy lại

Các lệnh dưới đây dùng test đã có trong repo. --no-cov giúp kiểm tra nhanh; khi CI yêu cầu coverage có thể bỏ tùy chọn đó.

```
.venv\Scripts\python.exe -m pytest -q --no-cov ^
tests\unit\test_documents.py ^
tests\unit\test_cross_format_content_identity.py ^
tests\unit\test_chunk_preembedding.py ^
tests\unit\test_ingestion_embedding_pipeline.py ^
tests\unit\test_ingestion_worker.py ^
tests\contract\test_knowledge_quality_migration.py ^
tests\contract\test_knowledge_quality_hardening_migration.py ^
tests\contract\test_chunk_preembedding_migration.py ^
tests\evaluation\test_knowledge_quality_benchmark.py
```

```
.venv\Scripts\python.exe -m pytest -q --no-cov ^
tests\unit\structured_facts ^
tests\integration\test_structured_fact_adapter.py ^
tests\integration\test_structured_fact_review_adapter.py ^
tests\end_to_end\test_structured_fact_review.py ^
tests\contract\test_structured_fact_layer_migration.py ^
tests\contract\test_structured_retrieval_filters_migration.py
```

Kết quả đã ghi nhận trong audit này

Duplicate focused: 99 passed; adapter/integration/evaluation bổ sung: 33 passed. Structured conflict focused: 101 passed. Retrieval/review audit: 84 passed. Các nhóm có overlap, vì vậy không cộng chúng thành một tổng test duy nhất.

Một lượt pytest rộng hơn đã chạy đến 67% mà chưa thấy failure nhưng bị timeout 180 giây; báo cáo không dùng lượt chưa hoàn tất đó làm bằng chứng pass toàn suite. Đây là cách ghi kết quả trung thực thay vì suy diễn từ progress.

5.3. Kịch bản demo bảo vệ: raw duplicate

1. Chuẩn bị một file UTF-8 hợp lệ dưới 10 MiB, ví dụ gia.txt.
2. Upload lần một, ghi lại document_id, object_path và số documents/object/jobs trong notebook.
3. Copy đúng bytes sang filename khác và upload lần hai.
4. Chứng minh response lần hai is_duplicate=true và document_id bằng lần một.
5. Query documents bằng owner/notebook/content_hash: chỉ một canonical active; Storage không có object path thứ hai; không có ingestion job thứ hai.
6. Chạy lại bằng hai request song song: một response tạo winner, response còn lại trả winner; không có hai canonical active.

```
SELECT id, owner_id, notebook_id, original_filename, content_hash,
       canonical_document_id, status, is_active, quality_status
FROM documents
WHERE owner_id = :owner AND notebook_id = :notebook
      AND content_hash = :sha256
ORDER BY created_at, id;
```

5.4. Kịch bản demo: cross-format canonical duplicate

1. Tạo TXT và DOCX có cùng visible prose đủ 40 ký tự/6 token, không ảnh/OCR.
2. Chứng minh SHA-256 raw của hai file khác nhau.
3. Upload/ingest TXT trước; chờ status ready và ghi normalized_content_hash/version.
4. Upload DOCX; vì raw hash khác, object và job mới được tạo; worker parse rồi H_canonical match.
5. Chứng minh DOCX row có canonical_document_id=TXT id, is_current=false, quality_status=duplicate; relation exact_content auto_confirmed confidence=1.
6. Chứng minh alias không có chunks/vectors/structured facts của riêng nó và job kết thúc duplicate_suppressed.
7. Đổi một table value hoặc row order để chứng minh hash không còn match. Thêm image chưa biểu diễn để chứng minh trust gate chặn auto-alias.

```
SELECT id, original_filename, content_hash, normalized_content_hash,
       normalization_version, canonical_document_id, is_current, quality_status
FROM documents
WHERE notebook_id = :notebook
ORDER BY created_at;
```

5.5. Kịch bản demo: chunk exact reuse

1. Ingest document A có một chunk dễ nhận biết; ghi strict hash, embedding_text_checksum, model và vector ID.
2. Ingest document B không exact document nhưng chứa nguyên chunk đó với cùng embedding context.
3. Bật quality mode on; quan sát dedup stats và metadata pre-embedding decision.
4. Chứng minh provider call không nhận chunk reusable và vector B bằng vector candidate A.
5. Đổi contextual prefix hoặc embedding model; chứng minh exact relation có thể còn nhưng vector không được reuse.
6. Dùng paraphrase gần giống; chứng minh relation near_duplicate pending, precomputed_vectors rỗng.

5.6. Kịch bản demo: structured conflict

Tập trước	Tập sau	Kết quả mong đợi
A101/S1, list price 4,5 tỷ, effective 01–31/08	A101/S1, list price 4,8 tỷ, effective 01–31/08	conflict_candidate pending
Như trên tháng 07	Như trên tháng 08	updated, không conflict
A101/S1	A101/S2	conditional_variant/source-target only
List price	Discounted price	conditional_variant
VAT included	Không nói VAT	uncertain
4.499.950.000 derived	4.500.000.000 published	unchanged/equivalent nếu tolerance

1. Upload bảng trước và bảng sau; bảo đảm header map có project/building/unit/price/effective time/qualifier.
2. Kiểm table_snapshots.row_count và structured_claims để chứng minh mọi row được extract, source cells/provenance đúng.
3. Kiểm claim_relations relation_type/review_status/reason và hai claim IDs.
4. Gửi chat hỏi đúng subject/time; chứng minh structured path trả warning và answer cite hai source chunks.
5. Mở review, confirm_conflict với reason; thử gửi lại expected_updated_at cũ để chứng minh 409 stale.

6. Dismiss một candidate khác và chứng minh generator không còn force citation pair cho relation dismissed.

5.7. Deployment gates còn phải chạy trên staging/live

Gate	Ca kiểm thử	Tiêu chí đạt
Migration order	Apply 08→09→10→16 trên snapshot dữ liệu thật	Không fail; backfill duplicate trước unique index; schema checksum đúng
RLS/tenant isolation	User A/B query/mutate documents/claims/relations	Không đọc/ghi chéo tenant; service RPC đúng quyền
Raw upload race	Nhiều client upload cùng bytes đồng thời	Một canonical; losers trả winner; không orphan objects/jobs
Normalized race	Ingest nhiều format cùng content đồng thời	Advisory lock/unique chọn một canonical; stale worker không commit
Lease fencing	Cho lease hết rồi worker khác reclaim	Token cũ không complete/replace vectors/facts
Qdrant crash windows	Crash trước/sau DB completion/finalize	Không phục vụ generation chưa authoritative; reconciliation dọn orphan/old
Structured race	Hai prior/current bảng ingest đồng thời	Reload + bounded replace tạo relations đầy đủ, không duplicate key sai
Review concurrency	Hai reviewer resolve cùng timestamp	Một thành công, một 409; audit đầy đủ
Performance	EXPLAIN ANALYZE hash/LSH/structured queries trên dữ liệu đại diện	Index được dùng; latency/candidate bound đạt SLO
Observability	Theo dõi exact reuse, fuzzy pending, conflict rate, retry/fail-open	Có dashboard/alert và detector_version drill-down

5.8. Các giới hạn còn tồn tại

Giới hạn	Tác động	Hướng xử lý/diễn đạt
Raw hash chỉ bắt cùng bytes	Khác encoding/format không match lớp 1	Canonical projection xử lý sau parse; chấp nhận chi phí upload/parse
Canonical trust gate bảo thủ	Có false negative với OCR/visual/short docs	Đúng chủ đích; review/candidate path thay vì auto-alias
Fuzzy threshold domain-dependent	SimHash/ANN có thể sinh nhiều/ít candidate	Đánh giá benchmark Việt/domain; không nói quyền auto-merge
Generic detector probe giới hạn	Không chứng minh mọi row bảng lớn	Structured full-table path là authoritative cho business tables
Header/row identity không đủ	Nhiều uncertain	Bổ sung semantic header mapping/domain schema; không zip theo vị trí

Giới hạn	Tác động	Hướng xử lý/diễn đạt
Thiếu effective time	Không phân biệt update/conflict	Fail closed/uncertain; yêu cầu metadata/row time
Không tự currency conversion	Cross-currency unresolved	Chỉ thêm khi có tỷ giá có thời điểm/authority/provenance
Authority không tự chọn winner	Cần reviewer/policy	Đây là guard đúng; có thể thêm policy sau comparability
Candidate loader ưu tiên prior ready+active+current+canonical	Không quét mọi historical snapshot	Ghi rõ phạm vi; mở rộng query có temporal policy nếu cần
Structured persistence fail-open	Facts có thể trễ so với vector RAG	Retry/reconciliation/monitoring; UI hiển thị freshness
Cross-format alias vẫn giữ object	Tiết kiệm compute nhưng chưa tối ưu storage	Retention/garbage policy chỉ sau audit và reversible rule
Chưa live DB proof	Không được tuyên bố production-ready tuyệt đối	Chạy toàn bộ staging gates ở 5.7
Frontend audit environment	Lần build mới bị Node 20/EPERM, package cần Node ≥22	Không dùng lần này làm bằng chứng frontend build; chạy lại đúng runtime

5.9. Đề xuất cải tiến theo ưu tiên

Ưu tiên	Cải tiến	Giá trị
P0	Live migration/RLS/concurrency/Qdrant crash-window test có artifact	Đóng khoảng trống lớn nhất giữa local evidence và production correctness
P0	Dashboard exact reuse, fuzzy candidates, conflict/uncertain, structured freshness và cleanup lag	Phát hiện drift/false-positive theo detector version
P1	Domain benchmark Việt cho canonical/fuzzy/structured table với adversarial cases	Hiệu chỉnh threshold/header mapping bằng dữ liệu thật
P1	Reconciliation job cho Qdrant generation và structured fail-open	Giảm orphan/stale derived data sau crash
P1	Policy freshness/currentness rõ cho historical structured claims	Tránh candidate scope quá hẹp/quá rộng theo nghiệp vụ
P2	Retention policy cho cross-format alias objects có audit/restore	Tối ưu storage mà không phá traceability
P2	Authority resolution policy sau comparability, có explainability	Hỗ trợ reviewer chọn preferred claim nhưng không lẫn identity

5.10. Kết luận trạng thái hiện tại

Đã giải quyết tốt ở mức **implementation + local verification**

Exact byte, concurrent winner, cross-format canonical identity, trust gate, chunk exact reuse, fuzzy review-only, multi-facet structured conflict, full-table diff, atomic persistence, structured retrieval, two-sided citation và optimistic review đều có code/test evidence.

Chưa được phép kết luận **production hoàn tất**

Chưa có artifact xác nhận migrations/RLS/RPC/race/crash windows trên staging/live Supabase-Qdrant. Đây là deployment verification gap, không phải thiếu thuật toán cốt lõi trong repo.

Phụ lục A. Evidence catalog bám sát repository

Mã evidence được dùng để truy vết nhanh khi trình bày. Line number là vị trí tại thời điểm chốt báo cáo và có thể dịch khi code tiếp tục thay đổi; tên file/hàm/test là anchor ổn định hơn.

Mã	Vị trí	Chứng minh
CF-01	app/knowledge_quality/application/claims.py:318-897	Generic claim extraction/alignment và quantity/unit/date/negation/modality differences.
CF-02	app/structured_facts/domain/models.py:71-604	Location/Product/Commercial/BusinessScope, qualifiers, temporal, authority và claim identities.
CF-03	app/structured_facts/application/scope.py:78-249	Directional business scope, qualifier compatibility và effective interval comparison.
CF-04	app/structured_facts/application/table_analyzer.py:242-917	Full-row table analysis, value normalization, row identity, qualifiers, authority và temporal.
CF-05	app/structured_facts/application/table_analyzer.py:620-744	Business row identity và low-confidence unresolved fallback.
CF-06	app/structured_facts/application/table_diff.py:51-396	O(n+m) map join, decision pipeline, duplicate-key ambiguity và value tolerance.
CF-07	app/structured_facts/application/comparison.py:54-287	Rehydrate prior claims, match table families và map relation status/payload.
CF-08	app/structured_facts/application/persistence.py:50-430	Deterministic snapshot/claim payload, hashes, source cells, provenance và citation guard.
CF-09	supabase/migrations/16_structured_fact_layer.sql:8-668	Snapshot/claim/relation/audit schema, constraints, indexes, composite tenant links và RLS.
CF-10	supabase/migrations/16_structured_fact_layer.sql:679-1549	Atomic structured replacement RPC, locks, worker-safe statuses và auditing.
CF-11	app/ingestion/application/worker.py:576-838	Structured write only after canonical completion, candidate diff, reload/reconciliation và fail-open retry.
CF-12	supabase/migrations/16_structured_fact_layer.sql:1675-1900	Owner-guarded structured search, qualifier/time filters, citable claims và relation warnings.
CF-13	app/chat/application/services.py:215-325, 553-817	Structured intent/search/candidate projection và structured-first fallback control flow.
CF-14	app/generation/adapters/openai_generator.py:84-220, 234-340	Relation-id conflict pairs, prompt signals và post-stream citation enforcement.
CF-15	app/api/routers/structured_facts.py:24-160	List/evidence/resolve endpoints, owner concealment và stale 409.
CF-16	app/structured_facts/adapters/postgrest_repository.py:186-373	Pending-only list, two-sided evidence validation và optimistic resolve RPC.
CF-17	frontend/src/components/documents/StructuredFactReviewPanel.jsx:45+	Hai phía, qualifiers, raw cells, time, authority và reason trong review UI.

Mã	Vị trí	Chứng minh
CF-18	tests/unit/structured_facts; tests/integration/test_structured_fact_adapter.py; tests/contract/test_structured_fact_layer_migration.py	Unit/adapters/contract evidence cho structured conflict layer.
DUP-01	app/documents/domain/models.py:13-25, 133-180	Giới hạn, whitelist, file signature/encoding và raw SHA-256.
DUP-02	app/api/routers/documents.py:101-110	Giới hạn tối đa 20 file cho một request.
DUP-03	app/documents/application/services.py:149-220	Pre-check, early-return, create metadata và retry concurrent winner.
DUP-04	app/documents/adapters/postgres_repository.py:64-96, 160-186	Tenant-scoped canonical lookup và ánh xạ unique conflict.
DUP-05	supabase/migrations/08_knowledge_quality.sql:354-361	Partial unique index byte identity.
DUP-06	app/pipeline/documents/application/content_identity.py:44-168	Typed projection, stable JSON, trust signals và parsed fingerprint.
DUP-07	app/knowledge_quality/application/analysis.py:71-86, 99-173	Strict/loose normalization, fingerprint và auto-identity gate.
DUP-08	app/ingestion/application/worker.py:359-434	Parse/fingerprint/lookup và early-return trước contextualize/embed.
DUP-09	app/ingestion/adapters/postgres_repository.py:206-236	Lookup normalized identity theo tenant/hash/version/status.
DUP-10	supabase/migrations/09_knowledge_quality_harden_index.sql:1090-1118, 1175-1604	Normalized unique invariant, advisory lock, atomic alias/relation/audit/job completion.
DUP-11	app/knowledge_quality/application/chunk_preembedding.py:93-396	Probe, plan, LSH bands, verification và vector reuse guards.
DUP-12	app/knowledge_quality/application/analysis.py:579-593	SimHash 3-token shingle, SHA-256 feature và 64-bit accumulator.
DUP-13	supabase/migrations/10_chunk_preembedding_dedup.sql:1-366	8 band indexes và tenant-scoped bounded candidate RPC.
DUP-14	app/pipeline/indexing/application/pipeline.py:433-530, 645-683	Provider filter, dependency vector resolution và persisted decision metadata.
DUP-15	app/knowledge_quality/application/detection.py:69-208	ANN document candidate, scope/coverage aggregation và full relation analyzer.
DUP-16	app/knowledge_quality/application/analysis.py:176-453	Lexical/template/claim/scope/critical-difference classifier.
DUP-17	app/retrieval/adapters/mmr_reranker.py:19-103	MMR diversity và exact group/checksum collapse.
DUP-18	tests/unit/test_documents.py; tests/unit/test_cross_format_content_identity.py; tests/unit/test_chunk_preembedding.py	Các test exact upload, cross-format identity, chunk dedup và fuzzy safety.
FLOW-01	app/documents/application/services.py:215-300	Storage failure cleanup và enqueue ambiguity reconciliation.

Mã	Vị trí	Chứng minh
FLOW-02	app/ingestion/application/worker.py:315-347, 502-570	Download/verify, lease renewal, external generation persistence và reconciliation.
FLOW-03	app/pipeline/indexing/adapters/vector_indexes.py:272-342	Qdrant generation delete/finalize theo document/version/ingestion_generation.
FLOW-04	app/ingestion/application/worker.py:734-838	Structured candidate→diff→atomic replace, reload race reconciliation và fail-open.
FLOW-05	app/structured_facts/domain/review.py:35-52	Review status và actions được chấp nhận.
SYS-01	app/ingestion/application/worker.py:315-585	Download → verify → prepare → canonical duplicate → contextualize → chunk dedup → embed → fenced completion → structured write.
SYS-02	app/ingestion/application/worker.py:1009-1013	Worker kiểm tra lại size và SHA-256 của object đã download.
SYS-03	app/structured_facts/application/persistence.py:50-92	Persistence batch deterministic và không lưu raw table dump thứ hai.
VER-01	tests/evaluation/reports/knowledge_quality_v1_report.json:250-259	Checked-in benchmark exact auto-reuse TP/TN/FP/FN và recall.
VER-02	tests/contract/test_knowledge_quality_hardening_migration.py	Static contracts cho claim fencing, normalized identity serialization và hardening SQL.
VER-03	tests/contract/test_structured_fact_layer_migration.py	Static contracts cho schema/RLS/service RPC/atomic structured replace/search/review.
VER-04	tests/end_to_end/test_document_upload.py; tests/end_to_end/test_structured_fact_review.py	API behavior với dependency overrides/fake repositories; không phải live Supabase E2E.

Phụ lục B. Kịch bản trình bày bảo vệ

B.1. Mạch trình bày 12–15 phút

Thời gian	Chủ đề	Thông điệp
Phút 0–1	Nêu bài toán	Duplicate không chỉ là cùng tên; conflict không chỉ là hai số khác nhau. Mục tiêu là đúng, tenant-safe, tiết kiệm embedding và có audit.
Phút 1–3	Lớp raw exact	Validate → SHA-256 bytes → scoped lookup → early-return; partial unique + retry winner đóng race.
Phút 3–5	Cross-format	Extraction khác canonical projection; typed text/table, stable JSON, strict hash/version và trust gate.
Phút 5–7	Chunk	Hai checksum, SimHash-LSH chỉ candidate, full verification; vector reuse chỉ cùng input/model; fuzzy không reuse.
Phút 7–10	Structured conflict	Mọi row → business identity → scope → qualifier → unit → effective time → value; O(n+m) diff.

Thời gian	Chủ đề	Thông điệp
Phút 10–12	Persistence/retrieval	Atomic RPC, claim_token fencing, structured-first, relation-id two-sided citation, optimistic review.
Phút 12–15	Bằng chứng/giới hạn	Chỉ code+test local đã chứng minh; live Supabase/Qdrant gates còn phải chạy.

B.2. Câu hỏi thường gặp và câu trả lời sâu

1. Canonical là gì?

Là biểu diễn chuẩn, deterministic để so sánh. Trong duplicate cross-format, canonical là typed JSON sequence; trong nhóm document, canonical document là row đại diện mà alias trỏ tới.

2. Tại sao không hash filename?

Filename là nhãn, không phải nội dung. Hash filename làm đổi tên tạo identity mới và bỏ sót duplicate. Repo chỉ validate/sanitize tên cho metadata/path.

3. Tại sao owner/notebook không băm chung với bytes?

Hash mô tả content; owner/notebook là authorization/query scope. Composite key tách rõ identity và isolation, đồng thời worker có thể dùng raw hash kiểm integrity.

4. SHA-256 có tìm file gần giống không?

Không. SHA-256 là exact digest; đổi một byte thường đổi toàn digest. Near candidate dùng SimHash/ANN rồi full verification.

5. Pre-check rồi còn cần unique index làm gì?

Hai request có thể cùng thấy chưa có. Unique index là invariant atomic tại DB; pre-check chỉ tối ưu latency.

6. Partial unique là gì?

Unique chỉ áp dụng row canonical active, không failed. Alias/archived/failed có thể giữ lịch sử mà không phá invariant.

7. Retry winner có insert lại không?

Không. Request thua unique conflict lookup lại canonical winner và trả is_duplicate=true. Nếu không tìm thấy, fail rõ ràng.

8. Extraction đã có text/table, tại sao cần projection?

Extraction chứa chi tiết phụ thuộc format/parser. Projection chọn semantics ổn định để hash, nhưng vẫn giữ type/order/boundary.

9. Vì sao kind=text và kind=table quan trọng?

Nó ngăn prose và bảng flatten thành cùng chuỗi. Exact identity phải bảo toàn loại cấu trúc.

10. Vì sao row order đổi thì document hash đổi?

Document exact identity bảo thủ; order có thể mang nghĩa. Row-order-independent matching thuộc structured table diff, không thuộc document exact.

11. So với tài liệu cũ có parse lại file cũ không?

Không trong đường lookup thường. New file parse thành hash/claims; old hash/claims được đọc từ documents/structured tables đã persist.

12. Canonical JSON lưu ở đâu?

Nó tồn tại trong RAM worker để hash. Repo persist normalized hash/version và quality metadata cần lookup; không có canonical-comparison temp table.

13. Cross-format duplicate có còn object không?

Có. Nó chỉ được phát hiện sau upload/parse nên source object và alias row được giữ để audit; chunks/vectors/facts mới bị suppress.

14. Hash collision xử lý thế nào?

SHA-256 collision thực tế cực thấp. Ở chunk path, code vẫn kiểm strict hash giống mà normalized text khác thì raise/fail closed.

15. SimHash khác SHA-256 ở đâu?

SHA-256 làm exact identity; SimHash bảo toàn độ gần theo features để sinh candidate. SimHash không có quyền merge.

16. LSH giải gì?

Giảm không gian tìm kiếm: trùng một trong tám band được đưa vào candidate set. Application vẫn kiểm Hamming và full relation.

17. Tại sao Hamming threshold 24 không đủ?

Ngưỡng chỉ nói chữ ký gần. Số, ngày, phủ định, scope có thể khác; analyze_text_relation phải xác minh critical differences.

18. Vì sao exact chunk chưa chắc reuse vector?

Embedding biểu diễn input provider. Contextual prefix hoặc model khác làm vector space/input khác; phải khớp embedding checksum và model.

19. Same-batch reuse làm thế nào?

Group theo normalization version + strict hash, xác minh text/checksum, embed representative một lần và dependency chunks lấy vector đó.

20. MMR có phải duplicate detector?

Không. MMR là reranker retrieval; collapse exact group rồi cân bằng relevance/diversity để tránh context lặp.

21. Hai giá khác nhau đã là conflict chưa?

Chưa. Cần cùng row identity, scope/qualifier tương thích, unit/currency/basis tương thích, effective time overlap và value khác vượt tolerance.

22. Tại sao publication time không dùng thay effective time?

Ngày phát hành không chắc là ngày giá có hiệu lực. Dùng sai sẽ biến update thành conflict hoặc ngược lại.

23. Stable qualifier và optional qualifier khác nhau thế nào?

Stable vào candidate key; optional không đổi candidate key nhưng vẫn được so. Missing optional một phía dẫn UNKNOWN, không giả equal.

24. Authority cao có tự thắng không?

Không. Authority chỉ là evidence sau comparability. Nguồn official về scope khác không thể thắng một claim không cùng đối tượng.

25. Row identity tránh A101 S1/S2 ra sao?

subject_key gồm project/building/unit hoặc composite product fields; project/unit đơn độc không đủ khi có ambiguity.

26. O(n+m) đến từ đâu?

Mỗi phía dựng hash map theo subject/predicate và stable qualifier, sau đó duyệt union keys; không cross-product $n \times m$.

27. Bảng 1.000 row có sampling không?

Không ở structured analyzer/diff. Test tạo đủ 1.000 claims và join đúng khi đảo row. Sampling chỉ ở generic ANN document candidate path.

28. Duplicate business key xử lý sao?

Không zip row theo vị trí. Giữ từng claim một phía thành uncertain, reason=duplicate_business_key.

29. Conflict được lưu và trả lời thế nào?

claim_relations pending giữ hai claim/snapshot. Structured search trả warnings; generator nhóm theo relation_id và enforce citations hai phía.

30. Reviewer đồng thời có ghi đè nhau không?

RPC yêu cầu expected_updated_at. Một reviewer update làm timestamp đổi; request cũ nhận 409 và phải refresh.

31. Worker cũ sau khi mất lease có commit được không?

Không nếu DB invariant hoạt động: claim_token mới mỗi reclaim; renew/completion RPC kiểm worker, token và lease trong transaction.

32. Đã production-ready chưa?

Core implementation và local tests mạnh. Chưa được gọi production-verified cho đến khi migrations/RLS/races/Qdrant crash windows chạy trên staging/live có artifact.

33. Hệ thống ưu tiên false positive hay false negative?

Đối với auto action, ưu tiên tránh false positive: exact/trust/tenant guards rất chặt. False negative được đẩy sang candidate/review để không mất dữ liệu.

Phụ lục C. Bản đồ file theo chủ đề

Chủ đề	File nên mở khi trình bày
Upload validation + raw hash	app/documents/domain/models.py; app/documents/application/services.py
Raw unique/race	app/documents/adapters/postgrest_repository.py; supabase/migrations/08_knowledge_quality.sql
Canonical projection	app/pipeline/documents/application/content_identity.py; app/knowledge_quality/application/analysis.py
Worker end-to-end	app/ingestion/application/worker.py; app/ingestion/adapters/postgrest_repository.py
Normalized identity RPC	supabase/migrations/09_knowledge_quality_hardening.sql
Chunk dedup + SimHash-LSH	app/knowledge_quality/application/chunk_preembedding.py; supabase/migrations/10_chunk_preembedding_dedup.sql
Embedding provider filter	app/pipeline/indexing/application/pipeline.py
Generic conflict	app/knowledge_quality/application/claims.py; app/knowledge_quality/application/analysis.py
Structured domain/scope	app/structured_facts/domain/models.py; app/structured_facts/application/scope.py
Table analyzer/diff	app/structured_facts/application/table_analyzer.py; app/structured_facts/application/table_diff.py

Chủ đề	File nên mở khi trình bày
Structured persistence	app/structured_facts/application/comparison.py; app/structured_facts/application/persistence.py; migration 16
Retrieval/generation	app/chat/application/services.py; app/generation/adapters/openai_generator.py
Review	app/api/routers/structured_facts.py; app/structured_facts/adapters/postgrest_repository.py; StructuredFactReviewPanel.jsx

Phụ lục D. Checklist kết luận đúng/sai

Không nên nói	Nên nói chính xác
'SHA-256 tìm được mọi duplicate.'	SHA-256 raw chỉ tìm byte-identical; canonical projection xử lý exact xuyên format; fuzzy là review candidate.
'Pre-check chống được upload đồng thời.'	Pre-check là đường nhanh; partial unique index + retry winner mới đóng race.
'Extraction và canonical giống nhau.'	Extraction tạo dữ liệu parser-rich; canonical projection tạo identity typed, stable và versioned.
'SimHash giống thì reuse vector.'	SimHash/LSH chỉ sinh candidate; vector reuse cần strict exact, cùng embedding checksum/model và mode on.
'Hai giá khác nhau là conflict.'	Chỉ conflict khi cùng identity/scope/qualifier/unit, effective time overlap và khác value vượt tolerance.
'Authority cao tự thắng.'	Authority là post-comparability evidence; reviewer/policy mới chọn preferred source.
'Không lưu tạm gì cả.'	Intermediate ở RAM; không có DB temp comparison table; Qdrant generation là persistent staging.
'Đã test production.'	Đã local unit/integration/E2E-fake/static-contract; live Supabase/Qdrant verification còn là deployment gate.

— Hết báo cáo —